

GAT AI Virtual Campus ## Complete Project Documentation **Project (repository / config name):** *AI Agent-Based Indoor Virtual Campus Tour and Query Assistant* **Institution:** Global Academy of Technology (GAT), Rajarajeshwari Nagar, Bengaluru **Tagline:** *Growing Ahead Of Time* **This document serves as:** complete project documentation · technical implementation guide · architecture explanation · AI/RAG explanation · virtual-campus explanation · algorithm reference · developer handover · team learning material · viva / interview preparation · future-development reference. **Method:** Every claim below was checked against the **current source code** in the repository. Where the code and the older design documents disagree, the code wins, and the disagreement is called out explicitly. Each capability is labelled: - **Implemented** — traceable to running source code. - **Partially implemented** — the mechanism exists but is incomplete, unused, or degraded. - **Documented but not currently implemented** — described in `CLAUDE.md` / `docs/architecture.md` but not present in code. - **Planned / Future Work** — explicitly deferred. - **Could not be verified from the current repository** — insufficient evidence. **No models, datasets, algorithms, sources, or performance numbers in this document are fabricated.**

Table of Contents

1. Executive Summary
2. Project Overview
3. Problem Statement
4. Motivation
5. Objectives
6. Proposed Approach
7. System Architecture
8. Technical Stack
9. Frontend Implementation
10. Backend Implementation
11. AI Assistant (Chatbot)
12. RAG Pipeline — Complete Explanation
13. Data Sources
14. LLM Model
15. Multi-Agent Architecture
16. Algorithms (Complete Catalogue)
17. 360° Virtual Campus
18. Virtual Tour Implementation
19. Navigation and A*
20. Database
21. Voice Assistant
22. API Architecture
23. End-to-End Data Flows
24. Implementation Decisions (Why Each Technology)
25. Project Uniqueness
26. Setup and Installation
27. Environment Variables
28. Testing
29. Performance
30. Security
31. Implementation Status Matrix
32. Limitations
33. Deployment Readiness
34. Future Development Roadmap
35. Viva Preparation
36. Beginner-Friendly Explanation
37. Source-Code Traceability Matrix
38. Conclusion - Appendix A: Diagrams Index - Appendix B: Documented-vs-Implemented Discrepancies

1. Executive Summary

GAT AI Virtual Campus is a single web platform that combines four things that a normal college website keeps separate:

1. **An AI question-answering assistant** for Global Academy of Technology. It answers in natural language, strictly from official GAT information (the college's own website pages and PDF documents), and refuses politely when it does not have a reliable answer. It runs entirely on a **local Large Language Model** (Meta **Llama 3.2** via **Ollama**) — no paid cloud AI service, no API key.
2. **An immersive 360° virtual tour** of the GAT Main Building — **real equirectangular 360° photographs** of the entrance and all four upper floors (156 scenes), navigable by clicking on-screen arrows, plus an automatic "guided walk" mode.
3. **A satellite campus map** (Google Maps satellite view) showing the campus location and its buildings.
4. **Voice interaction** — the user can ask questions by speaking (browser speech-to-text) and have answers read aloud (browser speech synthesis, tuned to a consistent male English voice).

The AI assistant is built on a **Retrieval-Augmented Generation (RAG)** pipeline: official GAT content is scraped, cleaned, split into ~1,500 text chunks, embedded into a local vector database, and — at query time — the most relevant chunks are retrieved (hybrid semantic + keyword search), reranked, confidence-scored, and handed to Llama 3.2 with strict "answer only from this context" instructions. A deterministic post-generation check verifies that specific facts (numbers, room numbers, phone numbers, dates) in the answer trace back to the retrieved sources; if they do not, the answer is withheld.

Routing to specialist "agents" (admissions / academics / facilities / navigation / general) is **deterministic** (keyword and phrase rules), with a small trained **PyTorch LSTM** intent classifier used only as a tie-breaker. A separate lightweight **small-talk layer** handles greetings, thanks and farewells instantly, without touching the retrieval pipeline.

Current state: the project runs locally as a full stack (Next.js frontend + FastAPI backend + PostgreSQL + ChromaDB + Ollama). It is a **college major-project / demo build**, not a deployed production service. There is no authentication, no cloud deployment, and no automated test suite in the conventional sense (tests exist as standalone verification scripts that require the live stack).

2. Project Overview

2.1 In plain language

Imagine you are a prospective student, a parent, or a visitor who wants to know something about Global Academy of Technology — "*What programmes do they offer?*", "*How do I apply?*", "*Where is the Chemistry lab?*", "*Is there a hostel?*" Today you would open the GAT website, guess which of dozens of pages or PDFs might contain the answer, scroll, and read. If you want to see the campus, you would look at a handful of photos.

GAT AI Virtual Campus replaces that with:

- **Ask a question, get an answer.** You type or speak a question in ordinary English (or Kannada / Hindi). The assistant reads the official GAT material for you and gives a short, direct answer, with the source pages it used.
- **Walk the building.** You open the Virtual Tour and move through real 360° photos of the Main Building — entrance, ground floor, first, second, third — by clicking arrows, exactly like Google Street View, or let it walk you through automatically.
- **See where the campus is.** A satellite map shows GAT's location and buildings.

2.2 Technical definition

GAT AI Virtual Campus is a **full-stack web application** implementing:

- a **multi-agent Retrieval-Augmented Generation (RAG) question-answering system** grounded in a locally-hosted knowledge base built from the official `gat.ac.in` website and its linked PDF documents, generated by a **local Meta Llama 3.2 model served through Ollama**, with deterministic grounding verification and confidence-gated refusal;
- a **WebGL 360° panorama tour** (Pannellum) over a **graph of real equirectangular photographs** stored as nodes/edges/panoramas in PostgreSQL, with manual and automatic ("guided") navigation and per-scene orientation calibration;
- a **graph-based campus pathfinding engine** (A* with a Dijkstra fallback) exposed as a REST endpoint;
- **browser-native voice input/output** (Web Speech API) with a deterministic voice-selection layer;
- a **PostgreSQL** relational store for campus structure, tour data, chat sessions, curated FAQ answers, and fee facts.

2.3 Target users

User	What they use
Prospective students & parents	AI assistant (admissions, fees, programmes, placements), Virtual Tour, Map
Current students & visitors	AI assistant (academics, facilities, "where is room X"), Virtual Tour
The GAT institution	A single, always-available front door to official information; a demonstrable AI project
The development team / evaluators	This document; the phased, independently-runnable codebase

2.4 Major features (as implemented)

Feature	Status
Conversational AI assistant grounded in official GAT content	Implemented
Local LLM (Ollama + Llama 3.2), no cloud AI	Implemented
Hybrid retrieval (semantic + BM25) + reranking + confidence scoring	Implemented
Deterministic grounding / hallucination check + graceful refusal	Implemented
Multi-agent deterministic routing + trained LSTM intent fallback	Implemented
Small-talk / greeting / gratitude / farewell layer	Implemented
Curated verified-FAQ fallback tier	Implemented
Structured fee knowledge base	Implemented
360° Virtual Tour of the Main Building (156 real scenes)	Implemented
Guided (automatic) tour walk-through	Implemented
Minimap, orientation calibration, cross-floor sightline hotspots	Implemented
Google satellite campus map with building markers	Implemented (needs an API key; degrades gracefully without)
Voice input (speech-to-text)	Implemented (browser-dependent)
Voice output (text-to-speech) with deterministic voice selection	Implemented (browser-dependent)
Multilingual UI + multilingual AI answers (EN / KN / HI)	Implemented
Persistent chat sessions + contextual follow-up resolution	Implemented
Campus A* pathfinding	Implemented in the backend / not currently reachable from the UI
RAGAS-style automated RAG evaluation	Documented but not implemented
Authentication / user accounts	Not implemented (deliberately)
Cloud deployment	Not implemented (local dev only)

3. Problem Statement

A conventional institutional website presents information as a large collection of static pages and downloadable PDF documents. For Global Academy of Technology specifically, the current project's knowledge base was built from **~60 website pages and ~135 PDF documents** on `gat.ac.in` (exam timetables, circulars, NIRF disclosures, brochures, department pages, admission documents, etc.). This creates concrete problems for a user who simply wants an answer:

Problem (only the ones this project actually addresses)	Evidence it is a real problem for GAT
Information is fragmented across dozens of pages and 100+ PDFs.	The knowledge pipeline had to crawl 60 pages and download 135 PDFs to cover common topics.
The user must know where to look. Answering "what is the fee for BE CSE?" requires finding the right admission document and reading a table.	Fee facts live in <code>Admission_2026.docx</code> , cross-checked against an AICTE approval letter — not on an obvious page.
No conversational interface. You cannot ask a question; you can only browse.	The base website has search at best, no Q&A.
No structured "where is X" answer. Room/department locations are not on the website at all.	Real Main-Building room numbers (112, 202, 203, 302, ...) exist only on physical signage, not in any GAT digital source.
Static, non-immersive campus view. A few photos, no way to "walk" the building.	The base site has no virtual tour.
No voice interaction / accessibility option.	The base site is text-only.
No single language switch that also covers spoken answers.	GAT's audience is largely Kannada/Hindi-speaking; the base site is English-only.

Problem → Our Solution

Problem	Our Solution (as implemented)
Information fragmented across many pages/PDFs	A RAG pipeline ingests all of it once, into one searchable vector store (<code>data/chroma_db</code> , collection <code>gat_kb</code> , ~1,507 chunks).
User must know where to look	Hybrid retrieval (semantic + keyword) finds the relevant chunks automatically; the user just asks.
No conversational interface	<code>POST /api/v1/chat</code> — natural-language question in, grounded answer + source list out.
Answers might be invented / outdated	Answers are generated only from retrieved official chunks; a deterministic grounding check (<code>scripts/ai/grounding.py</code>) withholds answers whose specific facts don't trace to the sources.
Low-relevance questions get a confident wrong answer	Confidence scoring (<code>scripts/ai/confidence.py</code>) routes low-confidence queries to a fixed refusal instead of the LLM.
"Where is room 202 / the CSE department?" not answerable from the website	A hand-verified spatial knowledge base (<code>data/campus_spatial/</code>) built by reading door signage in the real panoramas answers these, with the signage quoted as evidence.
Static campus view	156 real 360° photographs of the Main Building, navigable scene-to-scene (Pannellum + a nodes/edges graph).
No voice option	Browser Web Speech API for both input and output.
English-only	UI strings and spoken/written AI answers available in English, Kannada, Hindi.
Greetings ("hi", "thanks") wasted the retrieval pipeline and returned "no information"	A dedicated small-talk layer (<code>scripts/ai/smalltalk.py</code>) answers these instantly, before retrieval.

4. Motivation

The motivation is a combination of a **user-experience gap** and an **academic goal**.

User-experience gap. As Section 3 shows, GAT's official information is comprehensive but hard to *use*. A prospective student in a counselling queue, or a parent on a phone, does not want to open six PDFs. They want to ask and be answered — accurately, from official sources, without the answer being made up.

Academic goal. This is a final-year engineering major project. It is deliberately built to demonstrate, end to end and defensibly in a viva:

- a working **RAG pipeline** (ingestion → chunking → embedding → hybrid retrieval → reranking → grounded generation), not a thin wrapper around a hosted API;
- **local, private AI** — the entire language model runs on the developer's machine (Ollama + Llama 3.2), so there is no per-query cost, no data leaving the machine, and no API key to leak;
- **honest engineering** — every component that is a placeholder (untrained reranker, synthetic intent-classifier dataset, un-surveyed GPS coordinates) is labelled as such in the code and in this document, rather than dressed up;
- **systems integration** — a RAG chatbot, a 360° tour, a graph pathfinder, a voice layer and a relational database, integrated behind one frontend.

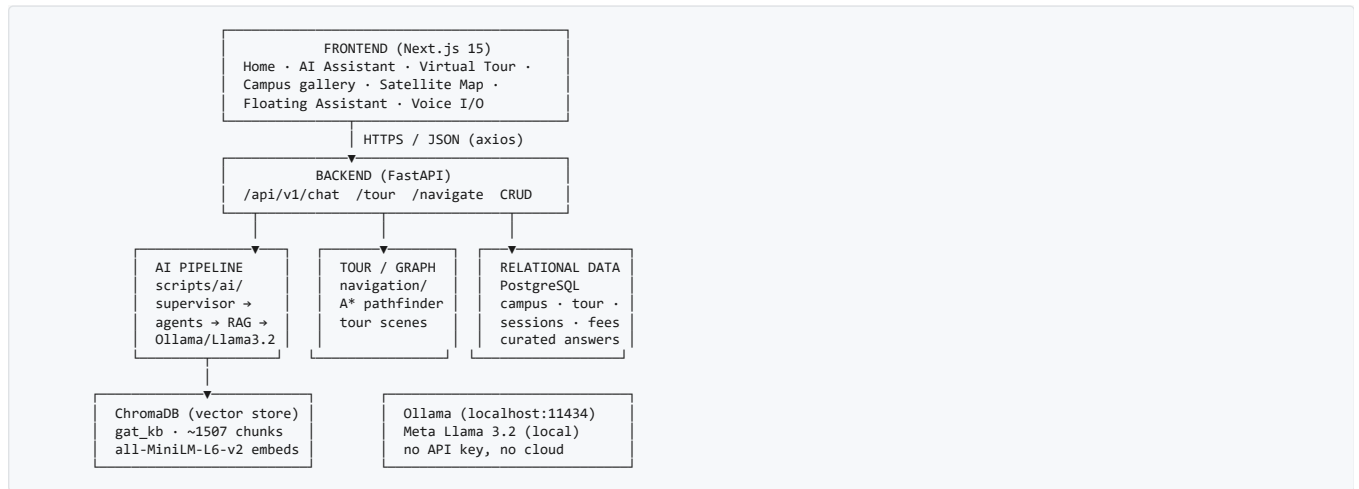
The project follows a **7-phase build plan** (`GAT_Virtual_Tour_Build_Guide.md`) that later expanded to ~19 numbered phases (visible in the git history: Phase 1 data acquisition → Phase 19 GPS navigation), each independently runnable and demoable.

5. Objectives

1. **Ground every AI answer in official GAT material.** No answer should contain a GAT-specific fact that cannot be traced to a retrieved chunk from `gat.ac.in`.
2. **Never fabricate.** When the knowledge base does not support a confident answer, refuse politely and point the user to the official website / institution.
3. **Keep the AI local and free to run.** Use Ollama + a small Meta Llama model; no external LLM API, no API key, no per-query cost.
4. **Make routing explainable.** Every routing decision (which agent handled a query) should trace to a literal keyword/phrase match or a logged classifier score.
5. **Build a real immersive tour** from real 360° photographs of the GAT Main Building, structured so that dropping in more/better photos later is a data change, not a code change.
6. **Provide graph-based campus navigation** (A*), correct and admissible, even where real coordinates are missing (degrade to Dijkstra).
7. **Support voice** for both input and output using only the browser, with graceful degradation on unsupported browsers.
8. **Support three languages** (English, Kannada, Hindi) across the UI and the AI's answers.
9. **Persist conversations** so that follow-up questions ("how do I get there?", "what about the AI course?") can be resolved against earlier turns.
10. **Stay honestly scoped** — a demoable college project, not an over-engineered production SaaS.

6. Proposed Approach

The system is built as **one platform integrating several independent subsystems**, each with a clean interface so it can be developed, tested and reasoned about on its own:



Why one platform? The three user-facing capabilities (ask, tour, locate) share data and context:

- the tour's **nodes/edges/panoramas** are the same PostgreSQL graph the pathfinder uses;
- the assistant's **"where is X"** answers come partly from the same panoramas (their signage);
- a conversation started in the **Floating Assistant** on any page continues seamlessly in the full `/chat` page (shared session + message store);
- the **language switch** applies to the UI, the speech recogniser, and the AI's answer at once.

Development approach. Strictly **phase by phase** (`CLAUDE.md` , `GAT_Virtual_Tour_Build_Guide.md`). Each phase is independently runnable and was verified by a standalone script (`scripts/ai/test_phaseN_.py`) before the next phase began. Placeholders are isolated behind stable interfaces ("swap the file, not the code").

7. System Architecture

7.1 Layered backend architecture

The backend follows a strict four-tier downward dependency (each tier calls only the tier below it):

TIER 1 – API LAYER	backend/app/api/v1/*.py
FastAPI routers. HTTP concerns only: validation, dependency wiring, response shaping. No business logic.	
chat.py · tour.py · navigate.py · crud_router.py (buildings, floors, rooms, nodes, edges, panoramas, cross-floor-hotspots, documents, campuses)	
TIER 2 – ORCHESTRATION	scripts/ai/supervisor.py + agents
Decisions: is this small talk? multi-domain? which agent?	
supervisor.route() → smalltalk / conversation_agent	
→ multi-domain split	
→ classify() → admission/academic/facilities/ navigation/general agent	
TIER 3 – DOMAIN SERVICES	
RAG:	hybrid_retrieval · reranker · context_selection · confidence · grounding · llm_generator · query_expansion
Intent:	backend/app/intent_model/ (PyTorch LSTM)
Navigation:	backend/app/navigation/ (A*, graph_builder, direction_formatter, room/building search)
Spatial:	scripts/ai/spatial_knowledge · campus_tools
Curated:	scripts/ai/curated_answers
TIER 4 – DATA / INFRA	
PostgreSQL (SQLAlchemy 2.0 + Alembic)	
ChromaDB PersistentClient (./data/chroma_db, collection gat_kb)	
Ollama HTTP (http://localhost:11434, model llama3.2)	
Flat JSON: data/campus_spatial/*.json, data/processed/chunks.jsonl	
Session store: backend/app/session/store.py (PostgreSQL-backed)	

Note on scripts/ai/ vs backend/app/ . The AI pipeline lives in scripts/ai/ (so each phase stayed independently runnable via python scripts/ai/...). The running FastAPI app imports it via a sys.path insertion in backend/app/api/v1/chat.py . scripts/ai/ in turn imports parts of backend/app/ (the navigation package, the intent model, the ORM models via campus_db.py). This bidirectional coupling is deliberate and documented in the module docstrings.

Async note. docs/architecture.md states "all I/O is async". In the current code the chat handler is a **synchronous** def (not async def) — deliberately, because the whole Phase 1–6 pipeline (ChromaDB, sentence-transformers, SQLAlchemy, the Ollama HTTP call) is synchronous blocking code; FastAPI runs sync handlers in a thread pool, so a slow request never blocks the event loop. Concurrent Ollama calls are bounded to 1 by a semaphore in llm_generator.py .

7.2 Chatbot request flow (actual)

```
User (types or speaks)
|
▼
Frontend ChatInput.tsx / SuggestedQuestions → useChatConversation.sendMessage()
| POST /api/v1/chat { message, session_id?, language? } (axios, 25s timeout)
|
▼
backend/app/api/v1/chat.py :: chat()
| 1. RESPONSE_LANGUAGE.set(language or "en")
| 2. get_or_create_session() (PostgreSQL, best-effort)
| 3. is_conversational = smalltalk.detect(message) is not None
| 4. if NOT conversational: Phase-15 contextual reference resolution
|   · "ambiguous" (2+ active entities) → return a clarifying question, STOP
|   · else → effective_message = resolved follow-up (or raw message)
|
▼
scripts/ai/supervisor.py :: route(effective_message)
| (0) smalltalk.detect() → conversation_agent (fixed reply, NO retrieval/DB/LLM)
| (1) detect_multi_domain() → split on "and"/";" → 2+ different agents? → run each
| (2) classify():
|   23 navigation phrases → navigation_agent
|   bare room-number regex → navigation_agent
|   4 domain keyword sets → best-scoring agent
|   LSTM classify_intent() (conf ≥ 0.6) → mapped agent
|   else → general_agent
|
▼
<agent>.handle(query)
| navigation_agent → spatial_knowledge lookup → campus_tools DB lookup → else RAG
| academic_agent → department/programme aggregation (deterministic) → else RAG
| admission/facilities/general → RAG directly
|
▼
agent_base.run_specialist(agent_name, query) [the shared RAG pipeline]
| hybrid_search() → dense (ChromaDB) + BM25, normalized, fused 0.6/0.4, top-20
| rerank() → heuristic weighted features, top-5
| apply_domain_boost() (+0.05 for the routing agent's known sources)
| select_context() → drop near-duplicates (Jaccard≥0.75), drop weak chunks (<0.35xtop)
| compute_confidence() → 0.4·intent_fallback + 0.6·retrieval → HIGH/MEDIUM/LOW
| generate_answer():
|   LOW confidence → fixed refusal string (LLM NOT called)
|   Ollama unreachable/model → typed error status
|   else → Ollama /chat with SYSTEM_PROMPT + CONTEXT block + QUESTION
|   → find_unsupported_claims(answer, context) (grounding check)
|   · unsupported number/phone/room/year → grounding_check_failed refusal
|   curated-answer fallback (cosine ≥ 0.55) if RAG produced no confident grounded answer
| naturalize_answer() – Llama rephrases verified tool/curated/aggregated text,
| re-checked both ways; any failure → exact template
|
▼
back through supervisor → chat.py
| persist user + assistant messages (PostgreSQL)
|
ChatResponse { answer, status, confidence, confidence_level, selected_agent,
|               tool_used, sources[], navigation?, panorama?, session_id }
|
▼
Frontend chatStore → ChatMessageBubble renders answer + sources
| if voice-submitted OR "read answers aloud" on → useSpeechSynthesis.speak(answer)
```

7.3 Virtual campus (tour) architecture

```
User opens /tour
| GET /api/v1/tour/scenes?building_id=<MAIN> (also: /floors, /cross-floor-hotspots)
|
▼
backend/app/api/v1/tour.py
| reads PostgreSQL: nodes (with panorama), edges (outgoing → hotspots), floors
| each SCENE = one Node that has a Panorama
| each HOTSPOT = one outgoing Edge (yaw, pitch, direction label, entry orientation)
|
▼
Frontend useTourPanoramas → buildPanoramaEngine (frontend/src/features/tour/engine)
| builds an in-memory graph of PanoramaNode objects with .next/.prev/.hotspots
|
▼
PanoramaViewer.tsx → Pannellum (pannellum-react, WebGL)
| loads /panoramas/main-building/<floor>/NN.jpg (4096x2048 equirectangular)
| renders hotspot arrows at (yaw,pitch); click → onHotspotClick(targetId)
| Manual Tour: user clicks arrows · Guided Tour: useGuidedTour walks the path
| Minimap, compass, "look behind" hold, orientation calibration (admin), fullscreen
|
▼
tourStore (zustand) keeps currentNodeId in sync (shared with minimap)
```

7.4 360° / panorama pipeline (build time)

```
Real 360° camera photos (equirectangular, one folder per floor)
| C:\Users\harsh\Desktop\campus photos\main building\{entrance, ground floor, ...}
|
▼
scripts/media/build_panoramas.py ← Pillow (PIL) ONLY. NO AI model.
| · EXIF orientation correction (ImageOps.exif_transpose)
| · resize to 4096 px wide (LANCZOS), JPEG q82, progressive → NN.jpg
| · resize to 960 px wide, JPEG q60 → NN-preview.jpg
| · write manifest.json { building, floors[ { slug, level, scenes[...] } ] }
|
▼
frontend/public/panoramas/main-building/<floor-slug>/NN.jpg (+ -preview.jpg)
|
▼
scripts/db/seed.py reads manifest.json → inserts Node + Panorama + sequential Edge rows
```

There is no AI/computer-vision model anywhere in this pipeline. No diffusion model, no depth estimation, no flat-image-to-panorama conversion, no image stitching. The source photographs are already 360° equirectangular images captured with a 360° camera; build_panoramas.py only resizes and compresses them. See Section 17 for the full investigation.

7.5 Voice input / output pipeline

```
VOICE INPUT (speech → text)                VOICE OUTPUT (text → speech)
Microphone                                  Assistant answer text (from /chat)
| getUserMedia preflight (acquireMicStream) | useSpeechSynthesis.speak()
↓                                           ↓
window.SpeechRecognition / webkitSpeechRecognition cleanTextForSpeech() (strip markdown/URLs)
| lang = SPEECH_LANG[uiLanguage] (en-IN/kn-IN/hi-IN) | splitIntoSpeechChunks()
| interim + final transcripts                        | waitForVoices() (robust async)
↓                                                   | selectPreferredVoice() (deterministic
transcript text                                    | male en-US hierarchy, localStorage)
| onSend(transcript, { viaVoice: true })           ↓
the SAME useChatConversation.sendMessage()         SpeechSynthesisUtterance
→ POST /api/v1/chat → RAG/LLM → answer              (rate 0.97, pitch 0.95, volume 1.0)
|                                                   → window.speechSynthesis.speak()
↓                                                   ↓
answer is auto-spoken because viaVoice=true        Speakers (per-utterance queue, cancel-on-new)
```

8. Technical Stack

Versions below are the **declared minimums** from `requirements.txt` / `frontend/package.json` (and, where checked live, the resolved version).

8.1 Frontend

Technology	Version (declared)	Purpose	Where used	Why
Next.js	^15.0.0 (running 15.5.x, Turbopack)	React framework, App Router, routing, dev server	<code>frontend/src/app/*</code>	Modern React SSR/routing standard; App Router for file-based routes
React	^19.0.0	UI library	all components	Team familiarity, ecosystem
TypeScript	^5.6.0 (<code>strict: true</code>)	Static typing	entire <code>frontend/src</code>	Catch errors at compile time; strict mode enforced
Tailwind CSS	^3.4.0	Utility-first styling	all components, <code>globals.css</code>	Fast, consistent styling without a component-library lock-in
zustand	^4.5.5	Client state stores	<code>chatStore</code> , <code>languageStore</code> , <code>tourStore</code> , <code>campusStore</code>	Minimal global state; <code>chatStore</code> persists to <code>localStorage</code>
@tanstack/react-query	^5.59.0	Server-state fetching/caching	<code>tour</code> + CRUD data hooks, <code>useChat</code>	Declarative fetch + cache
axios	^1.7.7	HTTP client	<code>frontend/src/api/client.ts</code>	Interceptors, per-endpoint timeout (chat = 25 s)
pannellum-react	^1.3.6	360° panorama WebGL viewer	<code>frontend/src/features/tour/PanoramaViewer.tsx</code>	Mature equirectangular viewer with hotspots, compass, autorotate
@vis.gl/react-google-maps	^1.9.0	Google Maps JS SDK React bindings	<code>frontend/src/features/mapSatellite/*</code>	Real satellite imagery of the campus (Phase 17)
framer-motion	^11.11.0	Animations / transitions	landing, Floating Assistant, tour panels	Declarative enter/exit animations
next-themes	^0.4.6	Light/dark theme	<code>providers.tsx</code>	Theme toggle (defaults light)
lucide-react	^0.451.0	Icon set	throughout	Consistent SVG icons
eslint / eslint-config-next	^9 / ^15	Linting	<code>npm run lint</code>	<code>next/core-web-vitals</code> + <code>next/typescript</code>

Not present in the current frontend: **MapLibre GL JS**, **Three.js** / **react-three-fiber**, **i18next**. (See Appendix B.)

8.2 Backend

Technology	Version	Purpose	Where
FastAPI	>=0.115	Web framework / routers / OpenAPI	<code>backend/app/main.py</code> , <code>backend/app/api/v1/*</code>
Uvicorn	>=0.32 ([standard])	ASGI server	run command
Pydantic	>=2.9	Request/response schemas, validation	<code>backend/app/schemas/*</code>
pydantic-settings	>=2.5	Env-driven config	<code>backend/app/core/config.py</code>
SQLAlchemy	>=2.0	ORM (<code>Mapped</code> , <code>mapped_column</code>)	<code>backend/app/models/*</code> , <code>backend/app/crud/*</code>
Alembic	>=1.13	DB migrations	<code>database/migrations/</code>
psycopg2-binary	>=2.9	PostgreSQL driver	<code>DATABASE_URL=postgresql+psycopg2://...</code>
python-dotenv	>=1.0	<code>.env</code> loading	<code>_shared.py</code> , <code>config</code>
httpx	>=0.27	HTTP client (Ollama probes)	<code>llm_generator.py</code> (via <code>ollama</code>)

8.3 AI / RAG stack

Technology	Version	Purpose	Where
Ollama (server + Python client)	server <code>ollama/ollama:latest</code> · client <code>>=0.3</code>	Local LLM runtime	<code>http://localhost:11434</code> ; <code>scripts/ai/llm_generator.py</code>
Meta Llama 3.2	<code>llama3.2</code> (resolves to <code>llama3.2:latest</code> , ~3.8 params, ~2 GB)	Grounded answer generation, answer naturalisation, multilingual output	configured by <code>OLLAMA_MODEL</code> ; called via <code>langchain_ollama.ChatOllama</code>
LangChain	<code>>=0.3</code> (<code>langchain</code> , <code>langchain-community</code> , <code>langchain-ollama</code>)	LLM call abstraction + text splitter	<code>ChatOllama</code> , <code>RecursiveCharacterTextSplitter</code>
ChromaDB	<code>>=0.5</code>	Vector store (embedded, persistent)	<code>chromadb.PersistentClient("./data/chroma_db")</code> , collection <code>gat_kb</code> , <code>hnsw:space=cosine</code>
sentence-transformers	<code>>=3.0</code>	Local embedding model runner	<code>SentenceTransformer("all-MiniLM-L6-v2")</code>
all-MiniLM-L6-v2	HF model	Embedding model — 384-dim, normalised, cosine	<code>build_embeddings.py</code> , <code>hybrid_retrieval.py</code> , <code>curated_answers.py</code>
rank_bm25	<code>>=0.2</code>	BM25 Okapi lexical retrieval	<code>scripts/ai/hybrid_retrieval.py</code>
scikit-learn	<code>>=1.5</code>	<code>sklearn.svm.SVR</code> reranking infrastructure (never trained)	<code>scripts/ai/reranker.py</code>
PyTorch (torch)	<code>>=2.0</code>	LSTM intent classifier	<code>backend/app/intent_model/model.py</code>
beautifulsoup4	<code>>=4.12</code>	HTML cleaning of scraped pages	<code>collect_website.py</code> , <code>clean_and_chunk.py</code>
pypdf	<code>>=5.0</code>	Per-page PDF text extraction	<code>collect_pdfs.py</code>
requests	transitive	Website / PDF crawling	<code>collect_website.py</code> , <code>collect_pdfs.py</code>

8.4 Media / other

Technology	Version	Purpose	Where
Pillow (PIL)	<code>>=11.0</code>	Resize/compress the real 360° source photos	<code>scripts/media/build_panoramas.py</code> (build-time only)
Web Speech API	browser-native	Speech recognition (STT) + speech synthesis (TTS)	<code>useSpeechRecognition.ts</code> , <code>useSpeechSynthesis.ts</code>
Google Maps Platform (Maps JS API)	—	Satellite campus map	<code>/map</code> ; needs <code>NEXT_PUBLIC_GOOGLE_MAPS_API_KEY</code>

8.5 Tooling / infra

Technology	Purpose
Docker Compose	4 services: <code>frontend</code> , <code>backend</code> , <code>db</code> (<code>postgres:16-alpine</code>), <code>ollama</code> (<code>docker-compose.yml</code>)
pytest / pytest-asyncio	declared; <code>testpaths=["tests/backend"]</code> currently empty (Section 28)
ruff / black / mypy	Python lint / format / type-check (<code>pyproject.toml</code> , line length 100, Python 3.12)
Python venv	<code>venv/</code> at repo root (backend + AI deps)
npm	frontend package manager
Git / GitHub	version control; <code>.github/workflows/</code> CI placeholders
LaTeX (IEEEtran)	<code>gat_virtual_campus_ieee/</code> — a separate IEEE-format research paper about the project

9. Frontend Implementation

Framework: Next.js 15 App Router, React 19, TypeScript (strict), Tailwind. Entry point: `frontend/src/app/layout.tsx` (root layout: `Providers` → `Navbar` → `page` → `Footer` → `FloatingAssistant`).

9.1 Routes (`frontend/src/app/`)

Route	File	Purpose
<code>/</code>	<code>page.tsx</code> + <code>features/landing/*</code>	Home — hero (video background), feature cards, campus showcase, leadership section
<code>/chat</code>	<code>chat/page.tsx</code>	Full-page AI Assistant (<code>ChatWindow</code>)
<code>/tour</code>	<code>tour/page.tsx</code>	360° Virtual Tour
<code>/map</code>	<code>map/page.tsx</code>	Google satellite campus map (client-only)
<code>/campus</code>	<code>campus/page.tsx</code>	Campus photo gallery / building list
<code>/campus/[buildingId]</code>	<code>campus/[buildingId]/page.tsx</code>	Per-building detail

Navbar links: Home · Campus · Virtual Tour · Map · AI Assistant.

9.2 Feature modules (`frontend/src/features/`)

Module	Key components
<code>chat/</code>	<code>ChatWindow</code> , <code>ChatInput</code> , <code>ChatMessageBubble</code> , <code>TypingIndicator</code> , <code>SuggestedQuestions</code> , <code>FloatingAssistant</code> , <code>MicDiagnosticPanel</code>
<code>tour/</code>	<code>PanoramaViewer</code> , <code>TourSidebar</code> , <code>TourControls</code> , <code>TourTopBar</code> , <code>TourModeToggle</code> , <code>Minimap</code> , <code>FloorSelector</code> , <code>GuidedTourControls</code> , <code>GuidedTourPanel</code> , <code>ImmersiveToggle</code> , <code>OrientationCalibrationPanel</code> , <code>CrossFloorHotspotPlacementPanel</code> , <code>engine/</code> (pure tour-graph logic)
<code>mapSatellite/</code>	<code>SatelliteCampusMap</code> , <code>GoogleSatelliteMap</code> , <code>BuildingMarker</code> , <code>CampusMarker</code> , <code>BuildingGeoInfoPanel</code> , <code>SatelliteMapUnavailable</code> , <code>campusLayout.ts</code>
<code>landing/</code>	<code>Hero</code> , <code>Features</code> , <code>CampusShowcase</code> , <code>leadership section</code>

9.3 Hooks (`frontend/src/hooks/`)

Hook	Responsibility
<code>useChat.ts</code> (<code>useChatSend</code>)	TanStack-Query mutation → <code>POST /api/v1/chat</code>
<code>useChatConversation.ts</code>	The one place a message enters the pipeline (typed input, suggested question, or voice transcript); manages <code>chatStore</code> + <code>sessionId</code>
<code>useSpeechRecognition.ts</code>	Speech-to-text: <code>SpeechRecognition</code> / <code>webkitSpeechRecognition</code> , mic preflight (<code>acquireMicStream</code> — OS default then every enumerated input device), press-to-talk + tap-to-toggle, per-code errors, 25 s cap
<code>useSpeechSynthesis.ts</code> + <code>voiceSelection.ts</code>	Text-to-speech: robust async voice loading, deterministic voice selection (Section 21), <code>localStorage</code> persistence, per-utterance queue, cancel-on-new
<code>useTourPanoramas</code> , <code>useGuidedTour</code> , <code>usePanoramaPreloader</code> , <code>useTourKeyboardShortcuts</code> , <code>useCrossFloorHotspots</code>	Tour data + guided walk + preloading + keyboard nav
<code>useBuildings/useFloors/useRooms/useNodes/useEdges/usePanoramas/useDocuments</code>	TanStack-Query wrappers over CRUD endpoints
<code>useTranslation.ts</code>	<code>t(key)</code> lookup against <code>lib/i18n/translations.ts</code> for the current language

9.4 State stores (`frontend/src/store/` , `zustand`)

Store	Fields	Persistence
<code>chatStore</code>	<code>messages[]</code> , <code>isAssistantTyping</code> , <code>voiceEnabled</code> , <code>sessionId</code>	<code>persist</code> → <code>localStorage["gat-chat"]</code>
<code>languageStore</code>	<code>language</code> (<code>en</code> / <code>kn</code> / <code>hi</code>)	persisted
<code>tourStore</code>	<code>currentNodeId</code> , <code>currentLocationName</code> , <code>guidedPathNodeIds</code>	in-memory
<code>campusStore</code>	<code>selectedBuildingId</code>	in-memory

9.5 API communication

`frontend/src/api/` is the only layer allowed to hit the network. `client.ts` builds the axios instance (`NEXT_PUBLIC_API_BASE_URL` , default `http://127.0.0.1:8000/api/v1`). `chat.ts` uses a 25 s timeout (the only endpoint that waits on the local LLM); the rest use the default.

9.6 User journey (assistant)

Any page → Navbar "AI Assistant" OR the Floating Assistant bubble
→ `/chat` or the widget shows the persisted welcome message + history
→ user types / taps mic in `ChatInput` (voice: press-hold or tap-to-toggle; interim transcript fills live)
→ `useChatConversation.sendMessage()` → sets "typing" → `POST /api/v1/chat`
→ backend RAG/LLM pipeline (Section 7.2) → { `answer` , `status` , `confidence` , `sources` , ... }
→ `ChatMessageBubble` renders answer + de-duplicated source list + a "listen" button
→ if asked by voice OR "read answers aloud" is on → `useSpeechSynthesis.speak(answer)`

9.7 Internationalisation

`frontend/src/lib/i18n/translations.ts` — a plain `{ key: string }` dictionary per language; `t(key)` returns the string or the key. This is **not** `i18next`. `SPEECH_LANG` maps `en` → `en-IN`, `kn` → `kn-IN`, `hi` → `hi-IN` for the speech recogniser. The AI's *answer* language is server-side: the requested language is threaded into the Llama system prompt.

10. Backend Implementation

Framework: FastAPI. **Entry point:** `backend/app/main.py`.

10.1 Startup

`configure_logging()` → `FastAPI(title=..., lifespan=lifespan)` → `CORSMiddleware` (explicit allow-list) → `register_exception_handlers` → `include_router(api_router, prefix="/api/v1")` → `GET /health`. On startup, `chat.warmup()` eagerly builds the BM25 index (~1,507 chunks), the `all-MiniLM-L6-v2` model, the ChromaDB handle, and preloads the Ollama model, so the first request pays no cold start.

10.2 API layer (`backend/app/api/v1/`)

- `chat.py` — `POST /api/v1/chat`; the only AI route; synchronous `def`. Session handling, Phase-15 contextual resolution, small-talk gating, then `supervisor.route()`. Persists both turns.
- `tour.py` — `GET /tour/scenes?building_id=...`; builds tour scenes from `nodes` + `panoramas` + `edges`.
- `navigate.py` — `GET /navigate?start_node_id=&destination_node_id=&accessible_only=`; A* route between two graph nodes (no current frontend caller).
- `crud_router.py` + per-resource routers — generic list/get (and write) over the ORM models.
- `deps.py` — `get_db()` dependency yielding a SQLAlchemy `Session`.

10.3 Configuration (`backend/app/core/config.py`)

`pydantic-settings` reads `.env` relative to the repo root. Key: `DATABASE_URL`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL` (default `llama3.2` — single source of truth, read identically by `scripts/ai/`), `CHROMA_PERSIST_DIR`, `CHROMA_COLLECTION_NAME` (`gat_kb`), `CORS_ALLOWED_ORIGINS` (explicit allow-list, never `*`).

10.4 Schemas / models / CRUD

`backend/app/schemas/*` (Pydantic), `backend/app/models/*` (13 SQLAlchemy models, Section 20), `backend/app/crud/*` (generic `base.py` + per-model), `backend/app/db/` (`engine/session/ Base`), `database/migrations/` (Alembic, 12 revision files).

10.5 Session management (`backend/app/session/store.py`)

PostgreSQL-backed (Phase 8). `get_or_create_session`, `record_message`, `get_recent_active_entities`, `get_last_exchange`, `get_last_location`. **No Redis. No authentication** — `session_id` is an opaque server UUID, no user concept.

10.6 Error handling

Every agent / retriever / LLM failure is caught at the agent boundary and degraded to a typed status. App-wide handlers: `SQLAlchemyError` → 503, other → 500, both with generic messages (no internal detail leaked).

11. AI Assistant (Chatbot)

11.1 Trace: "Where is the CSE department?"

1. **Frontend** — `ChatInput` → `sendMessage("Where is the CSE department?")` → `POST /api/v1/chat`.
2. **chat.py** — sets language; opens session. `smalltalk.detect()` → `None`. Phase-15 resolution: no follow-up cue → `independent`, message unchanged.
3. **supervisor.route()** — small talk `None`; multi-domain `None`; `classify()` — the phrase "where is" is in `NAVIGATION_PHRASES` → `navigation_agent`.
4. **navigation_agent.handle()** — location request → tool path. `spatial_knowledge.search_spatial(...)` finds the verified record `DEPT_CSE_FIRST_FLOOR` (evidence: an overhead directional sign in `main-building/first-floor/01.jpg` reading "*DEPT. OF CSE / DEPT. OF ISE / ... CLASS ROOMS : 202, 203, 203A, 213 ...*", confidence 0.95). Returns a `tool_resolved` answer with an inline `Evidence:` clause. Confidence is the record's real value, not 1.0.
5. **naturalize_answer()** — Llama rephrases the verified sentence; the result is grounding-checked both ways and against a "grounded hard token" guard (room numbers, `CSE`, `ISE` must survive), else the exact template is used. The `Evidence:` clause is preserved verbatim.
6. **chat.py** — persists both turns; stores `resolved_location = "CSE department"` for a later "*how do I get there?*".
7. **Response** — `{ answer, status:"tool_resolved", selected_agent:"navigation_agent", tool_used:"spatial_knowledge", sources:[], ... }`.
8. **Frontend** — renders; speaks if voice-submitted / "read aloud" on.

11.2 Trace: "What is the NAAC grade of GAT?"

`classify()` → keyword "naac" in `general_agent` set → `general_agent` → `run_specialist()` → full RAG pipeline (Section 12) → Llama generates a grounded answer from retrieved `gat.ac.in` chunks, with `sources` populated.

11.3 Conversational behaviour (`scripts/ai/smalltalk.py`) — Implemented

Runs first in `route()`. `detect(message)` → one of eight categories by regex patterns, only when the *entire* message (filler words removed) is conversational:

Category	Examples	English reply (also <code> </code> , <code> </code>)
greeting	hi, hii, hey there, hello assistant	"Hi! How can I help you today?"
greeting_morning/afternoon/evening	good morning / afternoon / evening	"Good morning! How can I help you?"
how_are_you	how are you, how's it going, hru	"I'm doing well, thank you! How can I help you with the campus?"
capabilities	what can you do, how can you help	GAT-assistant capability blurb
identity	who are you, what's your name	"I'm the GAT Virtual Campus Assistant, here to help you with information about Global Academy of Technology..."
nicety	nice to meet you	"Nice to meet you too!..."
gratitude	thanks, thank you, ty, thanks a lot	"You're welcome! I'm happy to help."
farewell	bye, goodbye, see you, take care	"Goodbye! Feel free to ask me whenever you need help with the campus."
acknowledgement	ok, okay, got it, alright	"Alright! Let me know if there's anything else..."

`generation_status = "conversational"`, `selected_agent = "conversation_agent"`, **no retrieval, no DB, no LLM call** (~30 ms measured). A native-script table also catches नमस्ते, धन्यवाद, ಧನ್ಯವಾದಗಳು. Strictness check: "*Hi, where is the library?*" is **not** small talk (a real question remains) — it routes normally.

11.4 Contextual follow-ups (`scripts/ai/conversation_context.py` , Phase 15) — Implemented

Before routing, `resolve_reference(message, active_entities)`: - **resolved** (one active entity) — follow-up rewritten to a self-contained query, routed normally. - **ambiguous** (2+ active entities) — a clarifying question returned immediately; pipeline never touched. - **no_context / independent** — falls through; Phase 8 fallback prepends the previous question for "which one is closest?"-style comparisons.

12. RAG Pipeline — Complete Explanation

12.1 Why RAG?

A raw LLM will confidently invent college-specific facts — fake HOD names, wrong fees, plausible-but-wrong dates. For an official-information assistant that is unacceptable. RAG fixes it: the model is given the exact official passages and told to answer *only* from them, and to say so when they don't cover the question. The knowledge lives in a store that can be updated without retraining.

12.2 The pipeline, stage by stage (only stages present in code)

BUILD TIME (scripts/ai/run_pipeline.py)
1. COLLECT WEBSITE collect_website.py – polite same-domain crawl of https://www.gat.ac.in/ (depth ≤ 2, ≤ 80 pages, 0.6 s delay). Rejects assets, external links, the 3rd-party OPAC subdomain. → data/raw/website/*.html + *.meta.json 2. COLLECT PDFs collect_pdf.py – downloads every gat.ac.in PDF discovered above (≤ 15 MB), per-page text via pypdf. → data/raw/pdfs/*.pdf + *.pages.json + *.meta.json 3. CLEAN + CHUNK clean_and_chunk.py • HTML: BeautifulSoup strips nav/header/footer/script/style/cookie/breadcrumb/sidebar; keeps headings + li/p/td/th; nearest preceding heading → chunk `section`. • PDF: drop lines on > half the pages (running headers/footers); chunk per page so `page` is exact. • Splitter: RecursiveCharacterTextSplitter, chunk_size = 800 chars, chunk_overlap = 120 chars, separators = ["\n\n", "\n", ".", ",", " "] • Dedup: SHA-256 of normalised text; first wins. • Tags: chunk_id, text, source_url, source_title, source_type (official_website official_pdf), section, page, document_name, department, collection_date, knowledge_category="official_institutional". → data/processed/chunks.jsonl (1,507 chunks) 4. EMBED + STORE build_embeddings.py • SentenceTransformer("all-MiniLM-L6-v2").encode(texts, normalize_embeddings=True) → 384-dim unit vectors • chromadb.PersistentClient("./data/chroma_db").get_or_create_collection("gat_kb", metadata={"hnsw:space": "cosine", "embedding_model": "..."}).upsert(ids, embeddings, documents, metadatas) (batch 64) → data/chroma_db/ (persistent; ~1,507 records)
QUERY TIME (agent_base.run_specialist)
5. QUERY EXPANSION query_expansion.expand_query(q) – small hand-curated symmetric synonym map. Widens ONLY the candidate pool; reranking / confidence / the LLM prompt use the user's exact words. Out-of-domain queries returned unchanged. 6. HYBRID RETRIEVAL hybrid_retrieval.hybrid_search(q, top_k=5) a. DENSE ChromaDB cosine on the MiniLM query embedding → top-20 {chunk_id: 1 – cosine_distance} b. BM25 rank_bm25.BM25Okapi over 1,507 chunks (lowercase alphanumeric; NO stemming/stopwords) → top-20 c. NORMALISE each set independently min-max → [0,1] d. FUSE hybrid = 0.6·dense_norm + 0.4·bm25_norm → sort → top-5 7. RERANK reranker.rerank(q, cands, top_k=5) – deterministic weighted feature sum (SVR class exists but is UNTRAINED → heuristic fallback always runs): 0.55·hybrid_score + 0.25·query_term_coverage + 0.10·exact_phrase_match + 0.10·length_score 8. DOMAIN BOOST context_selection.apply_domain_boost – +0.05 to chunks from the routing agent's known-domain sources. 9. CONTEXT SELECTION context_selection.select_context – drop near-duplicates (token-Jaccard ≥ 0.75 vs a kept chunk); drop weak chunks (< 0.35 × top score). 10. CONFIDENCE confidence.compute_confidence(selected, q) 0.4·intent_component + 0.6·retrieval_component intent_component = retrieval_component + FALLBACK (LSTM P(intent) NOT wired in here – see 12.4) retrieval_component = 0.7·top1_rerank + 0.3·agreement HIGH ≥ 0.58 · MEDIUM ≥ 0.48 · LOW < 0.48 (9-query fit; 12.6) 11. GENERATE llm_generator.generate_answer(q, ctx, confidence) • no context → fixed refusal, LLM NOT called • confidence LOW → fixed refusal, LLM NOT called • Ollama down / model gone → typed error status • else → ChatOllama(model="llama3.2").invoke([SystemMessage(SYSTEM_PROMPT [+MEDIUM addendum] [+language instruction]), HumanMessage("CONTEXT:\n<numbered chunks + source titles>\n\nQUESTION: <q>\n\nAnswer using only the CONTEXT above."))] – width-1 semaphore, 60 s timeout 12. GROUNDING CHECK grounding.find_unsupported_claims(answer, ctx) Deterministic. phone / currency / room-number / year patterns in the answer; every ≥3-digit run in a matched claim must appear among the CONTEXT's digit-runs (so "Rs. 50,000" == "INR 50000"). Any unsupported claim → generation_status "grounding_check_failed" + withheld message. 13. CURATED FALLBACK agent_base – only if RAG produced no confident grounded answer: curated_answers.find_curated_answer – cosine similarity (same MiniLM embeddings) vs 7 hand-authored FAQ rows; returned only if similarity ≥ 0.55. 14. NATURALISE llm_generator.naturalize_answer – verified tool/curated/aggregated text → Llama rephrase → guards (bidirectional find_unsupported_claims, spelled-out number words, invented ≥3-digit numbers, dropped grounded hard tokens in any language, English lexical drift, required-entity preservation). Any failure → exact verified template. Never raises.

12.3 Where the data comes from — see Section 13.

12.4 Honest note — the intent classifier and confidence

confidence.py weights an intent_component at 0.4, **documented** to be the LSTM's P(intent). In the current code, run_specialist() calls compute_confidence(selected_context, query) **without** an intent_probability, so intent_component falls back to *equalling* the retrieval component. The trained LSTM's probability is used for **routing** (supervisor.py) but **is not fed into the confidence score**. This is a genuine gap between the design doc and the code.

12.5 The system prompt (`llm_generator.SYSTEM_PROMPT`)

"You are the GAT Virtual Campus Assistant ... using ONLY the official GAT context supplied to you below. Answer only using the supplied CONTEXT. Do not invent GAT facts. Do not assume information that is not present ... Never invent faculty names, phone numbers, departments, fees, timings, locations, rules, courses, or facilities ... Preserve important factual details (numbers, names, dates) exactly ... Do not mention chunk IDs, retrieval scores, or internal system details."

A MEDIUM-confidence addendum tells the model to hedge. For KN/HI, a language instruction is appended ("respond in natural fluent Kannada/Hindi; keep proper nouns — GAT, VTU, CSE, ISE, ... — untranslated").

12.6 Confidence thresholds — how they were set

From `confidence.py`'s own comments: computed confidence for 6 in-domain GAT questions + 3 deliberately out-of-domain ("population of Bangalore?", "capital of France?", "bake a chocolate cake?"). Observed: out-of-domain 0.4392–0.4687; in-domain 0.4824–0.6907. `MEDIUM` = 0.48 sits in that gap; `HIGH` = 0.58 in a second gap inside the in-domain cluster. **Fitted to one 9-query sample, not a validated boundary.**

12.7 What happens when nothing relevant is found / confidence is low

Situation	Result
No retrieved context	<code>no_context</code> — fixed message pointing to <code>gat.ac.in</code> . LLM not called.
Confidence LOW	<code>low_confidence_refusal</code> — the fixed <code>LOW_CONFIDENCE_MESSAGE</code> . LLM not called.
LLM's own answer hedges even at HIGH	Curated-answer fallback consulted.
A specific fact can't be verified	<code>grounding_check_failed</code> — the answer is withheld.
Ollama down / model missing	<code>ollama_unreachable</code> / <code>model_unavailable</code> — clear typed message.

13. Data Sources

Question: where did the chatbot's knowledge come from?

Answer: entirely from the official Global Academy of Technology website, <https://www.gat.ac.in/>, **and the PDF documents linked from it.** This is verified in code (`scripts/ai/_shared.py`: `OFFICIAL_DOMAIN = "gat.ac.in"`, `OFFICIAL_BASE_URL = "https://www.gat.ac.in/"`) and in the collected artefacts (`data/raw/website/`, `data/raw/pdfs/`, `data/metadata/source_manifest.json`).

Correction of documentation. CLAUDE.md Says "`scripts/build_kb.py` relabels a Kaggle CSV into GAT-branded content." **This is not what the current code does.** There is no `scripts/build_kb.py` and no Kaggle CSV anywhere in the repository. The actual pipeline is `collect_website.py` + `collect_pdfs.py` → `clean_and_chunk.py` → `build_embeddings.py`, working on **real scraped GAT content**. Status: *the Kaggle-CSV description is documented but not implemented.*

13.1 The sources

Source	What it is	Data extracted	Used by	Location in repo
https://www.gat.ac.in/ website	~60 real HTML pages: About/NEF, admission, undergraduate & postgraduate programme pages (CSE, ISE, ECE, EEE, ME, CE, Aeronautical, AI&ML, AI&DS, MBA, M.Tech CSE, M.Tech Structural), faculty, placements, library, NAAC, NBA, NIRF, IQAC, phd-program, cells, campus-life, contact-us, examinations, academic-calendar, etc.	Cleaned heading/list/paragraph/table text, with the nearest heading kept as <code>section</code> and department inferred from the URL slug	RAG (all specialist agents)	<code>data/raw/website/*.html</code> + <code>*.meta.json</code> ; <code>chunk metadata source_type = "official_website"</code>
gat.ac.in PDF documents	~135 PDFs: exam & make-up-exam timetables, absentee/malpractice/CRV circulars, NIRF data sheets (2022–2025, engineering/management/overall), campus brochure, EoA report, MoUs, autonomous exam regulations, department course-structure (DCS) sheets, NPTEL course lists, cell notifications, VTU recognition/autonomy letters, etc.	Per-page extracted text (pypdf), running headers/footers removed, chunked per page (<code>page metadata exact</code>)	RAG	<code>data/raw/pdfs/*.pdf</code> + <code>*.pages.json</code> + <code>*.meta.json</code> ; <code>source_type = "official_pdf"</code>
source_manifest.json	The provenance ledger — one entry per source URL with <code>status</code> (<code>collected</code> → <code>chunked</code> → <code>embedded</code> or <code>failed</code>) and <code>chunk_count</code>	Proof of "where the AI knowledge came from"	audit / traceability	<code>data/metadata/source_manifest.json</code>
data/processed/chunks.jsonl	The single flat file of 1,507 chunks with full metadata	The BM25 index source + the ChromaDB documents	<code>hybrid_retrieval.py</code> , <code>build_embeddings.py</code> , <code>academic_agent.py</code>	<code>data/processed/chunks.jsonl</code>
fee_information table (PostgreSQL)	16 rows of structured fee facts	The authoritative fee record; the RAG fee chunks are <i>generated from</i> these rows	<code>admission_agent</code> (indirectly, via chunks)	<code>backend/app/models/fee_information.py</code> ; seeded by <code>scripts/db/seed_fee_data.py</code>
curated_answers table (PostgreSQL)	7 manually-authored verified FAQ Q/A pairs (admission-office contact, admission timings, entrance exams, UG courses list, PG courses list, MCA availability, hostel)	A fallback tier below RAG when confidence is low	<code>agent_base.run_specialist()</code>	<code>backend/app/models/curated_answer.py</code> ; seeded by <code>scripts/db/seed_curated_answers.py</code> + <code>scripts/db/curated_answer_questions.py</code>
data/campus_spatial/*.json (Phase 9)	Hand-verified spatial records built by reading door/directional signage in the real panoramas	room / department / laboratory / facility / landmark → floor + panorama file + a literal transcription of the sign + a confidence score	<code>navigation_agent</code> (via <code>spatial_knowledge.py</code>)	<code>data/campus_spatial/spatial_knowledge.json</code> (+ <code>department_locations.json</code> , <code>room_locations.json</code> , <code>laboratory_locations.json</code> , <code>facility_locations.json</code> , <code>landmarks.json</code> , <code>panorama_relationships.json</code> , <code>phase9_report.md</code>)
documents table (PostgreSQL)	5 rows — a plain relational content store (title / body / domain)	Not wired into retrieval; a legacy/skeleton store	—	<code>backend/app/models/document.py</code>

13.2 Fee-data sourcing (from `docs/fee_and_curated_answers.md`)

Fee amounts in `fee_information` come from GAT's own official document "*Admission 2026.docx*" (linked from <https://www.gat.ac.in/admission.html> as "Fee Structure"), cross-checked against **AICTE's Extension of Approval letter for AY 2026-27**. Where an amount is not publicly disclosed, the row records `amount = NULL` with a `notes` explanation — **never a guessed figure**.

13.3 Source → Processing → Storage → Retrieval → LLM

```
gat.ac.in HTML pages ┌
gat.ac.in PDF documents ┤ collect_website.py / collect_pdfs.py (crawl + download + pypdf)
                        ▼
                        data/raw/{website,pdfs}/* + source_manifest.json
                        ▼ clean_and_chunk.py (BeautifulSoup clean · header/footer strip ·
                        | RecursiveCharacterTextSplitter 800/120 · SHA-256 dedup · metadata tag)
                        data/processed/chunks.jsonl (1,507 chunks)
                        ▼ build_embeddings.py (all-MiniLM-L6-v2, normalized)
                        ChromaDB "gat_kb" (data/chroma_db/, cosine, ~1,507 vectors)

user query ──────────┬─ hybrid_retrieval (dense ChromaDB + BM25 over chunks.jsonl,
                        │                 normalized, fused 0.6/0.4, top-5)
                        │                 ▼ reranker (heuristic) → context_selection → confidence
                        top ~3-5 chunks + confidence category
                        ▼ llm_generator.generate_answer
                        CONTEXT block + SYSTEM_PROMPT + QUESTION
                        ▼ Ollama / Llama 3.2 (local)
                        grounded answer → grounding.find_unsupported_claims → final answer + sources[]
```

13.4 Could not be verified

- The **exact list of every URL** crawled is in `source_manifest.json`; a full enumeration is out of scope for this document but every source is a `gat.ac.in` URL.
- Whether every one of the 135 PDFs contributed usable text: some are image-only/scanned and were recorded as `failed` ("no extractable text — likely scanned/image-only PDF") by `clean_and_chunk.py`.

14. LLM Model

14.1 The exact model

Property	Value (verified)
Model	Meta Llama 3.2
Tag / identifier	llama3.2 (from OLLAMA_MODEL); ollama list on the dev machine shows llama3.2:latest
Approx. size	~3 billion parameters, ~2 GB on disk (the default llama3.2 tag is the 3B instruct model)
Provider / runtime	Ollama , running locally at http://localhost:11434 (OLLAMA_BASE_URL)
Cloud?	No . No OpenAI, no Anthropic/Claude API, no Google Gemini, no Groq, no HuggingFace hosted inference. No API key anywhere.
Where configured	.env / .env.example (OLLAMA_MODEL=llama3.2), backend/app/core/config.py (OLLAMA_MODEL: str = "llama3.2"), scripts/ai/llm_generator.py and scripts/ai/agent_base.py (os.environ.get("OLLAMA_MODEL", "llama3.2")) — one env var, read identically in all four places
How the app calls it	langchain_ollama.ChatOllama(base_url=..., model="llama3.2", client_kwargs={"timeout": 60}) .invoke([SystemMessage, HumanMessage]); availability first probed with the ollama Python client's .list()
Concurrency	bounded to 1 concurrent generation by threading.Semaphore in llm_generator.py (CPU-only host; two concurrent calls degraded badly in testing)
Fallback models	none — if llama3.2 is unavailable, the request returns model_unavailable; the model is never silently substituted
Gemini / other cloud model	Not implemented . Some earlier design notes and the user's build prompts mention "Meta AI / Gemini"; the code uses only local Ollama + Llama 3.2.

CLAUDE.md and README.md say "Meta Llama 3.2 (local)". docs/architecture.md (older) says "Claude API" — that document's own preamble marks it as superseded by Ollama/Llama. **The code uses Ollama + Llama 3.2, confirmed.**

14.2 The four jobs Llama 3.2 does

- Grounded answer generation** — the main RAG path (generate_answer): given the CONTEXT block + system prompt, produce an answer strictly from the context.
- Answer naturalisation** (Phase A/B/C, naturalize_answer) — rephrase an already-verified deterministic/curated/aggregated answer to read conversationally, then re-verify it did not change any fact.
- Multilingual output** — when the request language is `kn / hi`, the same model produces the answer in that language (system-prompt instruction).
- (Not implemented)* an isolated "LLM judge" for RAG evaluation — described in CLAUDE.md Phase 7, no `eval/` directory exists.

14.3 How the model fits the RAG architecture

```
retrieved + reranked chunks
↓
CONTEXT block ("[1] (Source: GAT - Admission)\n<chunk text>\n\n[2] (Source: ...)\n...")
↓
[ SystemMessage: SYSTEM_PROMPT (+ MEDIUM hedge addendum) (+ KN/HI language instruction) ]
[ HumanMessage: "CONTEXT:\n<block>\n\nQUESTION: <user question>\n\nAnswer using only the CONTEXT above." ]
↓
ChatOllama("llama3.2").invoke(...) ← width-1 semaphore, 60 s timeout, never raises past the boundary
↓
raw answer text
↓
grounding.find_unsupported_claims(answer, CONTEXT) ← deterministic, no second LLM call
↓
grounded answer OR "grounding_check_failed" (answer withheld)
```

The LLM is **never** the source of truth for a GAT fact. It is a language layer over verified retrieved text, wrapped in deterministic pre-checks (confidence gate) and post-checks (grounding).

15. Multi-Agent Architecture

15.1 What "multi-agent" means here

It is a **Supervisor + specialist-agents routing design**, hand-rolled in Python (`scripts/ai/supervisor.py`). It is **not** LangGraph, not a framework, and **not** the concurrency mechanism. All specialist agents run the **same** RAG pipeline (`agent_base.run_specialist()`); what differs is *which queries the Supervisor sends to each*, plus a couple of agents having a deterministic pre-RAG shortcut.

15.2 The agents

Agent	File	Purpose	Pre-RAG shortcut	LLM?	Retrieval?
conversation_agent	smalltalk.py (virtual)	greetings / thanks / farewell / small talk	is the whole handler	no	no
admission_agent	admission_agent.py	admission process, eligibility, application, fees	none	yes (RAG)	yes
academic_agent	academic_agent.py	departments, programmes, courses, curriculum, faculty	department/programme list aggregation (enumerates the real <code>*engineering*.html</code> pages from <code>chunks.jsonl</code>)	yes (RAG or naturalised aggregate)	yes
facilities_agent	facilities_agent.py	labs, classrooms, auditorium, library, hostel, canteen, transport, gym	attaches a <code>resolved_location</code> hint from <code>campus_tools.resolve_location()</code> when the query names one entity	yes (RAG)	yes
navigation_agent	navigation_agent.py	"where is X" — building / floor / room / department / lab locations; panorama lookups	spatial_knowledge (panorama signage) → campus_tools (PostgreSQL) → panorama lookup	only for the RAG fall-through	for the RAG fall-through
general_agent	general_agent.py	institution info, contact, history, accreditation, and the default fallback	none	yes (RAG)	yes

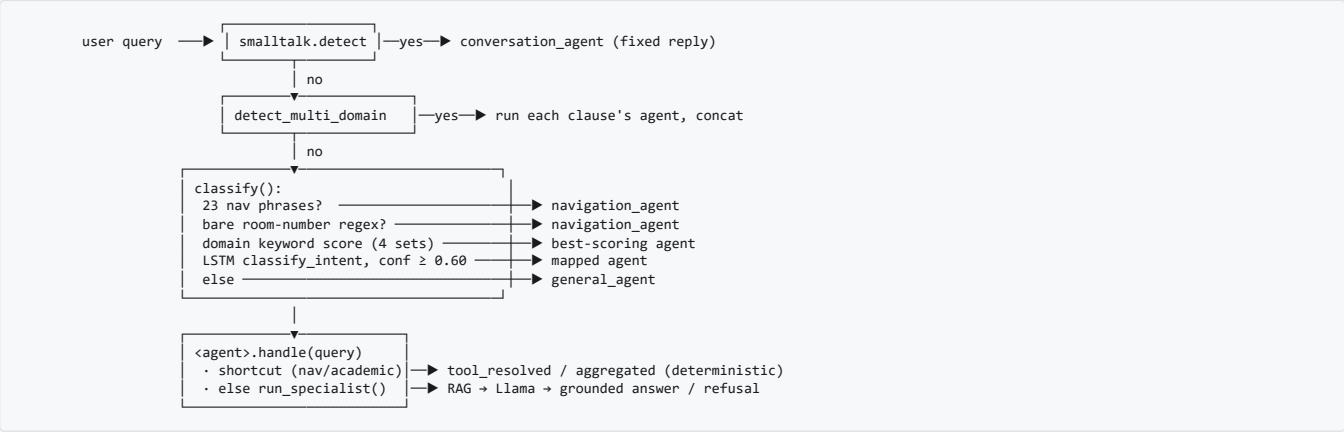
navigation_agent does NOT do route-finding. Phase 9 removed the "how do I get from X to Y" sub-intent and the `navigation_tool (A*)` call from the chat pipeline. Such a query now falls through to ordinary grounded RAG (never a fabricated route). The A* engine itself is untouched — see Section 19.

15.3 How a query is routed (`supervisor.route()` → `classify()`)

```
route(query):
    (0) conversational? smalltalk.detect(query) is not None
        → conversation_agent (fixed reply; STOP)
    (1) multi-domain? split on " and " / " ; " → 2+ clauses that classify to 2+ DIFFERENT agents
        → run each clause through its agent, concatenate answers (generation_status="multi_domain")
    (2) classify(query):
        a. for phrase in NAVIGATION_PHRASES (23 phrases: "where is", "which floor",
            "how do i reach", "take me to", "panorama", "in the virtual tour", ...):
            if phrase in query.lower(): return navigation_agent
        b. if a bare room-number regex ("room 202", "room no. 303") matches: return navigation_agent
        c. score each of 4 DOMAIN_KEYWORDS sets against the query:
            admission_agent : admission, apply, kcet, comedk, pgcet, cutoff, fee, fees, ...
            academic_agent  : department, course, program, curriculum, syllabus, faculty, b.e., m.tech, ...
            facilities_agent : facility, lab, laboratory, classroom, auditorium, library, hostel,
                                canteen, transport, bus, gym, wifi, ...
            general_agent   : global academy, about gat, contact, phone, email, naac, vision, ...
        → return the highest-scoring agent (if any keyword matched)
    d. RNN FALLBACK: classify_intent(query) via the trained LSTM
        if intent is not None and confidence ≥ 0.60:
            map the fine-grained intent → an agent (e.g. ROOM_LOCATION → navigation_agent)
            return that agent
    e. else: return general_agent (the confidence gate INSIDE the pipeline keeps unrelated
        queries safe, not this routing choice)
```

Every decision is logged with a human-readable reason ("matched navigation phrase 'where is'", "no domain keywords matched; RNN intent classifier predicted CAMPUS_INFO (confidence=0.72) → general_agent").

15.4 Agent-routing diagram



15.5 The LSTM intent classifier — Implemented (used as a routing fallback only)

Property	Value
Architecture	Embedding(vocab, 32) → LSTM(32→32, 1 layer) → Dropout(0.2) → Linear(32→12) (backend/app/intent_model/model.py)
Classes (12)	ADMISSIONS, COURSES, DEPARTMENTS, ACADEMICS, FACILITIES, ROOM_LOCATION, DEPARTMENT_LOCATION, LABORATORY_LOCATION, VIRTUAL_TOUR, CAMPUS_INFO, GENERAL, UNKNOWN
Training data	168 hand-authored synthetic examples (dataset.py) — <i>not</i> real user queries (this project has no query logs). Explicitly labelled a "bootstrap dataset".
Training	python -m app.intent_model.train — Adam, lr 0.01, weight_decay 1e-3, 150 epochs, seeded 80/20 split (134 train / 34 val), best-val-accuracy checkpoint
Artefacts	backend/app/intent_model/artifacts/ — intent_lstm.pt (~75 KB), vocab.json (283 tokens), intents.json , training_metadata.json
Measured held-out accuracy	0.5294 (18/34) on the validation split — a real, reproduced number (evaluation/intent_classification_report.txt ; macro-F1 0.4042, weighted-F1 0.4869). Some classes (LABORATORY_LOCATION, CAMPUS_INFO, GENERAL, UNKNOWN) scored 0 F1 on the tiny val set.
Role in the system	Consulted only when no deterministic phrase/keyword rule matched, and its prediction is used only if confidence ≥ 0.60 . It is a safety net for paraphrases outside the keyword lists (e.g. "What can I study here?"), never the primary router.
Degradation	If the artefacts are missing, classify_intent() returns intent=None, confidence=0.0 — treated as "no opinion", never a crash.

CLAUDE.md describes the classifier as 5 classes (admissions|academics|facilities|navigation|general). The **actual model has 12 fine-grained classes** that collapse onto the 5 agents.

16. Algorithms (Complete Catalogue)

Every meaningful algorithmic component found in the repository. Format per item: purpose · I/O · steps · formula (if any) · complexity · source · example.

16.1 Algorithm summary table

#	Algorithm	Purpose	Source file	Function / class
1	HTML content extraction	strip boilerplate, keep real text + headings	scripts/ai/clean_and_chunk.py	clean_html, strip_repeated_lines
2	Recursive character chunking	split docs into ~800-char overlapping chunks	clean_and_chunk.py (LangChain)	RecursiveCharacterTextSplitter, chunk_text_with_sections
3	SHA-256 content de-duplication	drop identical chunks	clean_and_chunk.py	inline hashlib.sha256
4	Dense embedding	text → 384-dim unit vector	build_embeddings.py, hybrid_retrieval.py	SentenceTransformer.encode(normalize=True)
5	Approximate NN vector search (HNSW, cosine)	top-k semantically similar chunks	ChromaDB	collection.query
6	BM25 Okapi lexical retrieval	top-k keyword-relevant chunks	hybrid_retrieval.py (rank_bm25)	BM25Okapi.get_scores
7	Min-max score normalisation	put dense & BM25 on one [0,1] scale	hybrid_retrieval.py	HybridRetriever._normalize
8	Weighted score fusion	one ranked list from two methods	hybrid_retrieval.py	HybridRetriever.hybrid_search
9	Deterministic feature reranking	reorder top candidates by precision signals	reranker.py	heuristic_rerank_score, extract_features
10	(SVR reranking — infrastructure only, untrained)	learned reranking (future)	reranker.py	SVRReranker
11	Query expansion	widen the candidate pool with synonyms	query_expansion.py	expand_query
12	Domain boost	nudge same-domain chunks up	context_selection.py	apply_domain_boost
13	Near-duplicate + weak-chunk filtering	clean the final context set	context_selection.py	select_context, _jaccard
14	Retrieval confidence scoring	pre-generation reliability estimate	confidence.py	compute_confidence
15	Grounding / hallucination check	verify answer facts trace to context	grounding.py	find_unsupported_claims
16	Answer naturalisation guards	verified text → LLM rephrase → re-verify	llm_generator.py	naturalize_answer + _english_answer_drifts, _missing_grounded_tokens, _introduced_digit_runs
17	LSTM intent classification	query → one of 12 intents + P	backend/app/intent_model/	IntentLSTM, classify_intent
18	Deterministic supervisor routing	query → specialist agent	supervisor.py	classify, route, detect_multi_domain
19	Small-talk intent detection	is this a greeting/thanks/...?	smalltalk.py	detect
20	Curated-answer semantic matching	query ≈ a verified FAQ?	curated_answers.py	find_curated_answer (cosine)
21	Department/programme list aggregation	enumerate all real department pages	academic_agent.py	_aggregate_departments
22	Spatial-knowledge lookup + room-number extraction	"where is room 202 / the CSE dept?"	spatial_knowledge.py	search_spatial, extract_room_number
23	Contextual reference resolution	rewrite a follow-up into a self-contained query	conversation_context.py	resolve_reference, _reformulate
24	A* shortest path	shortest walking route between two campus nodes	backend/app/navigation/pathfinding.py	find_shortest_path, _heuristic
25	Dijkstra (single-source)	nearest-neighbour distances (nearest panorama/room)	pathfinding.py	single_source_distances
26	Campus graph construction	DB rows → in-memory adjacency list	graph_builder.py	build_graph
27	Turn-by-turn direction formatting	path → human instructions	direction_formatter.py	format_directions
28	Tour scene-graph construction	DB nodes/edges/panoramas → walkable graph	tour.py + frontend ../engine	_build_scene, buildPanoramaEngine
29	Guided-tour traversal	automatically walk a floor's scene sequence	frontend/src/hooks/useGuidedTour.ts	useGuidedTour
30	Deterministic TTS voice selection	pick one consistent male en-US voice	frontend/src/hooks/voiceSelection.ts	selectPreferredVoice
31	Deterministic building placement	DB node coords → map/marker positions	frontend ../mapSatellite/campusLayout.ts	deriveBuildingPlacements

16.2 Selected algorithms in detail

Algorithm 8 — Weighted hybrid score fusion

- **Purpose:** combine semantic (embedding) and lexical (BM25) relevance into one ranking, so a query is answered by chunks that are both *about* the topic and *contain* its terms.
- **Input:** query `q`; the chunk corpus (1,507 chunks) + the ChromaDB collection.
- **Output:** top-5 chunks with `{semantic_score, bm25_score, normalized_*, hybrid_score}`.
- **Steps:** (1) `q' = expand_query(q)`; (2) dense: ChromaDB cosine $\rightarrow$ top-20 `{id: 1-dist}`; (3) BM25: `BM25Okapi.get_scores(tokens(q'))` $\rightarrow$ top-20 (drop zero-overlap); (4) min-max normalise each set independently; (5) for each candidate id, `hybrid = 0.6·dense_norm + 0.4·bm25_norm` (0 where absent from a method); (6) sort desc, take 5.
- **Formula:** `hybrid_score(c) = w_d · ndense(c) + w_b · nbm25(c)`, `w_d = 0.6`, `w_b = 0.4`.
- **Complexity:** dense query $O(\log N)$ (HNSW) + $O(\text{cand} \cdot d)$ scoring; BM25 $O(N \cdot |q|)$ per query (index built once at startup, $O(N \cdot L)$); fusion $O(\text{cand})$. `N = 1507`, `d = 384`.
- **Example:** "*Which branches does the college have?*" — dense finds the department pages semantically; BM25 catches the literal token "branches"; fusion keeps the department-listing chunks at the top.

Algorithm 9 — Deterministic feature reranking (heuristic fallback)

- **Purpose:** reorder the fused candidates by *answer-passage* quality signals that fusion alone misses (does the passage actually contain the query terms verbatim?).
- **Input:** query, the ~5–20 fused candidates.
- **Output:** top-5 with `rerank_score`, `rerank_mode`, `ranking_features`.
- **Features (all deterministic, [0,1] where used):** `hybrid_score`; `query_term_coverage = |qndoc tokens| / |q tokens|`; `exact_phrase_match` $\in \{0,1\}$ (normalised query is a substring of the normalised doc); `length_score = min(1, len(text)/400)`.
- **Formula:** `rerank_score = 0.55·hybrid_score + 0.25·query_term_coverage + 0.10·exact_phrase_match + 0.10·length_score` (weights sum to 1).
- **Note:** `SVRReranker` (RBF `sklearn.svm.SVR`) exists with working `fit/predict/save/load`, but **is never trained** — no labelled (query, chunk) $\rightarrow$ relevance dataset exists, and the code refuses to fabricate one. So `Reranker.mode` is always "heuristic_fallback".
- **Complexity:** $O(\text{cand} \cdot |q| \cdot L)$ for feature extraction.

Algorithm 14 — Retrieval confidence scoring

- **Purpose:** decide, *before* calling the LLM, whether the retrieved evidence is strong enough to attempt an answer.
- **Input:** the reranked+selected context list; the query.
- **Output:** `{confidence` $\in [0,1]$, `category` $\in \{\text{HIGH}, \text{MEDIUM}, \text{LOW}\}, \dots\}.$
- **Formula:** `confidence = 0.4 · intent_component + 0.6 · retrieval_component` `retrieval_component = clip(0.7 · top1 + 0.3 · agreement, 0, 1)` where `top1` = best chunk's `rerank_score`, and `agreement = max(0, top1 - mean(max(0, top1 - s_i)))` over the 2nd/3rd chunks (small gaps $\Rightarrow$ corroboration $\Rightarrow$ higher). `intent_component = retrieval_component` (documented fallback — the LSTM `P(intent)` is not wired in here).
- **Thresholds:** `HIGH` ≥ 0.58 , `MEDIUM` ≥ 0.48 , else `LOW` (fitted to a 9-query sample).
- **Effect:** `LOW` $\rightarrow$ fixed refusal, the LLM is never called.

Algorithm 15 — Grounding / hallucination check

- **Purpose:** after generation, catch a specific fabricated fact.
- **Input:** the generated answer; the exact CONTEXT block.
- **Output:** `{claim_type: [unsupported substrings]}` — empty dict = clean.
- **Steps:** (1) regex-extract candidate *number-bearing* claims from the answer — `phone ( \b(?:\+?91[-\s])?[6-9]\d{9}\b )`, `currency ( (?::rs\.|?|inr|₹)\s*[\d,]+ )`, `room number ( \broom\s*(?:no\.|#)?\s*\d{2,4}[a-z]? )`, `year ( \b(?:19|20)\d{2}\b )`; (2) reduce each match and the whole CONTEXT to the set of digit-runs of length ≥ 3 (commas stripped, so `50,000 == 50000`); (3) a claim is *unsupported* if none of its digit-runs appear in the CONTEXT's digit-runs.
- **Effect:** any unsupported claim $\rightarrow$ `generation_status = "grounding_check_failed"`, the answer is replaced with a "withheld" message.
- **Honest limitation (from the code):** this catches unsupported *numbers/phones/rooms/years*. A *wrong sentence* with no matchable numeric pattern (e.g. an invented HOD *name*) is not caught by this check — the system prompt + context grounding + confidence gate are what mitigate that.

Algorithm 24 — A* shortest path (backend/app/navigation/pathfinding.py)

- **Purpose:** shortest walking route between two campus graph nodes.
- **Input:** `CampusGraph` (adjacency list + `nodes_by_id`), `start_id`, `goal_id`.
- **Output:** `PathResult(node_ids, edges, total_distance)  $\rightarrow$  estimated_walk_time_minutes` (sum of per-edge `walking_time`, Or `total_distance / 1.2 m·s-1 / 60`), `is_accessible`.
- **Cost model:** `g(n)` = sum of edge `distance` values along the path so far. `h(n)` = straight-line distance to the goal: Euclidean on `pos_x/pos_y` if both nodes have them, else a small-scale planar approximation on `latitude/longitude` (`dx =  $\Delta\text{lon} \cdot 111320 \cdot \cos(\text{lat})$` , `dy =  $\Delta\text{lat} \cdot 110540$`), else `0`. `f(n) = g(n) + h(n)`.
- **Admissibility:** straight-line distance never overestimates real walking distance, so A* returns the optimal path. When `h = 0` (mixed/absent coordinate frames), A* degrades **exactly to Dijkstra** — still optimal, just no directional guidance. No separate Dijkstra code path is needed for that fallback.
- **Pseudocode (as implemented):** `find_shortest_path(graph, start, goal):` if `start == goal:` return trivial path `open` $\leftarrow$ min-heap of `(0.0, start)` `came_from` $\leftarrow \{\}$; `g` $\leftarrow \{\text{start: } 0.0\}$; `visited` $\leftarrow \{\}$ while `open:` `(_, current)  $\leftarrow$  pop-min(open)` if `current` $\in$ `visited:` continue `visited.add(current)` if `current == goal:` return `reconstruct(came_from, start, goal, g[goal])` for `e` in `graph.adjacency[current]:` # `GraphEdge(neighbor_id, distance, edge)` `tentative` $\leftarrow$ `g[current] + e.distance` if `tentative < g.get(e.neighbor_id,  $\infty$ ):` `g[e.neighbor_id]  $\leftarrow$  tentative` `came_from[e.neighbor_id]  $\leftarrow$  (current, e.edge)` `f` $\leftarrow$ `tentative + heuristic(nodes[e.neighbor_id], nodes[goal])` `push(open, (f, e.neighbor_id))` raise `NoPathFoundError`
- **Complexity:** $O(E \log V)$ with a binary heap (`heapq`). `V = 180` nodes, `E = 325` edges currently. Space $O(V + E)$.
- **Where used:** `GET /api/v1/navigate` only. **Not called by the chatbot. No current frontend caller** (the GPS-navigation UI was removed in commit `b3219d9`).

Algorithm 30 — Deterministic TTS voice selection (frontend/src/hooks/voiceSelection.ts , added this project cycle)

- **Purpose:** always speak with the *same* male English voice across page visits (`speechSynthesis.getVoices()` is unordered and populated asynchronously, so a naive pick flips male/female between visits).
- **Input:** the live `SpeechSynthesisVoice[]`; the target language; a `localStorage` -persisted previous choice.
- **Output:** one `SpeechSynthesisVoice` (or `null` → browser default).
- **Steps:** (1) if a persisted voice is still installed (match by `voiceURI`, then `name + lang`) → reuse it; (2) else evaluate a fixed priority hierarchy as a **pure function of the voice set** (sorted once by `natural-first` → `voiceURI` → `name`, so ties are order-independent): known male en-US names (`Microsoft Guy Online (Natural)` → ... `David` → ... `Mark` → `macOS Alex/Aaron/Fred/...`) → any en-US voice a male-name table marks male → any English male → any en-US → en-GB → any English → the sorted first voice (never `voices[0]`); (3) cache + persist the winner. Female-name table wins ties (fail-safe).
- **Utterance config:** `lang` = the chosen voice's own locale, `rate` = `0.97`, `pitch` = `0.95`, `volume` = `1.0`.
- **Verification:** 16 pure unit tests (`frontend/scripts/test-voice-selection.mts`) — identical pick across 8 shuffles per simulated platform (Windows/Edge, macOS, Chrome-Linux, minimal).

17. 360° Virtual Campus

This section answers the user's central question: *"How did we take flat images and create the 360° view?"*

17.1 The direct answer

We did not convert flat images into 360° images. The source images are **real equirectangular 360° photographs** captured with a 360° camera, one folder per floor of the GAT Main Building. The build step only **resizes and compresses** them.

No dedicated AI model for converting flat images into 360° panoramas was found in the current implementation. Specifically, the repository contains **no** diffusion model, **no** Stable Diffusion / img2img, **no** depth-estimation model (MiDaS / ZoeDepth / DPT), **no** panorama-generation model, **no** image-stitching code, and **no** call to any hosted image model or HuggingFace image endpoint. (Verified by searching the entire repository for: `pannellum`, `equirectangular`, `stable diffusion`, `diffusion`, `img2img`, `depth estimation`, `MiDaS`, `panorama generat`, `image stitching`, `spherical projection`, `Three.js` — the only hits are Pannellum for viewing, the `"projection": "equirectangular"` label in the panorama inventory JSON, and mentions in the design docs / IEEE paper.)

17.2 The image pipeline (build time) — verified

Field	Value
What the images are	Equirectangular 360° photographs (2:1 aspect ratio). The processed primaries are exactly 4096 × 2048 px .
Are they normal photographs?	No — they are 360° camera captures (spherical), stored in the equirectangular projection.
Are they generated / AI-transformed?	No.
Model used	None.
Library	Pillow (PIL) >= 11.0 only.
Source file	<code>scripts/media/build_panoramas.py</code>
Function	<code>build()</code> → <code>_process_photo()</code>
Input	JPEG files in a fixed 5-folder layout under <code>C:\Users\harsh\Desktop\campus photos\main building\</code> — <code>entrance</code> , <code>ground floor</code> , <code>first floor</code> , <code>second floor</code> (folder name kept verbatim as the user's source of truth), <code>third floor</code>
Processing	<code>ImageOps.exif_transpose</code> (fix orientation) → <code>convert("RGB")</code> → <code>resize(4096 wide, LANCZOS)</code> → JPEG q82 progressive → <code>NN.jpg</code> ; also <code>resize(960 wide)</code> → JPEG q60 → <code>NN-preview.jpg</code> (blur-up placeholder)
Output	<code>frontend/public/panoramas/main-building/<floor-slug>/NN.jpg</code> + <code>NN-preview.jpg</code> + <code>manifest.json</code>
Role in the system	Web-ready asset preparation. Not used by the running app; run once, offline.

17.3 The rendering pipeline (run time) — how the 360° experience works

Field	Value
Library	Pannellum (via <code>pannellum-react</code> ^1.3.6), loaded client-only (<code>next/dynamic</code> , <code>ssr:false</code>)
Technique	WebGL renders the equirectangular JPEG as the inside of a sphere; the camera sits at the centre; drag / arrow keys / touch rotate the view (yaw/pitch), scroll zooms (hfov)
Source file	<code>frontend/src/features/tour/PanoramaViewer.tsx</code>
Per-scene config	<code>image</code> (primary JPEG), <code>preview</code> (blur-up), <code>yaw / pitch / hfov</code> (calibrated opening view), <code>compass</code> , <code>autoRotate</code>
Hotspots	custom DOM markers at (<code>yaw</code> , <code>pitch</code>) — Street-View-style chevron arrows (<code>forward/back/left/right</code>), stair/elevator glyphs, and subtle "cross-floor sightline" arrows; click → navigate to the target scene
Scene transition	a brief 2 px-blurred veil of the destination's preview image, lifted the instant the new scene reports loaded (not a dual-canvas crossfade — 2× GPU cost for marginal gain)
Cleanup	the outgoing Pannellum instance's WebGL context + document/window listeners are explicitly <code>destroy()</code> -ed on every scene change (pannellum-react has no unmount hook)

17.4 How scenes are connected & how the user moves

- Data structure:** a **directed graph** stored in PostgreSQL — `nodes` (one per photographed spot), `edges` (one per navigable connection, with `yaw / hotspot_pitch` for the arrow, `direction`, `entry_yaw / entry_pitch` for the camera angle on arrival), and `panoramas` (image path + scene metadata, linked to a node).
- Backend** `GET /api/v1/tour/scenes?building_id=<MAIN>` returns each scene (= a node + its panorama) plus its hotspots (= its outgoing edges).
- Frontend** `buildPanoramaEngine` (`features/tour/engine`) turns that into an in-memory graph of `PanoramaNode` objects with `.next` / `.prev` / `.hotspots` references.
- Manual Tour:** the user clicks an arrow → `onHotspotClick(targetId)` → the page switches `currentNode` and Pannellum remounts with the new image; `entry_yaw` opens the new scene facing the direction of travel.
- Guided Tour:** `useGuidedTour` walks the floor's scene sequence automatically — for each step it eases the camera (look left, look right, look forward via `rotateTo`), then advances through the same `onAdvance` path Manual Tour uses (so Guided never bypasses the normal transition). Pausable / resumable / speed-adjustable.
- Cross-floor:** `cross_floor_hotspots` (849 rows) are hand-placed sightline markers ("that staircase leads to the 2nd floor") — rendered only on the exact panorama they were authored on, separate from the walking-sequence edges.
- Minimap:** a white-themed top-down mini-map of the current floor with a live heading marker, reading `currentNodeId` from `tourStore`.

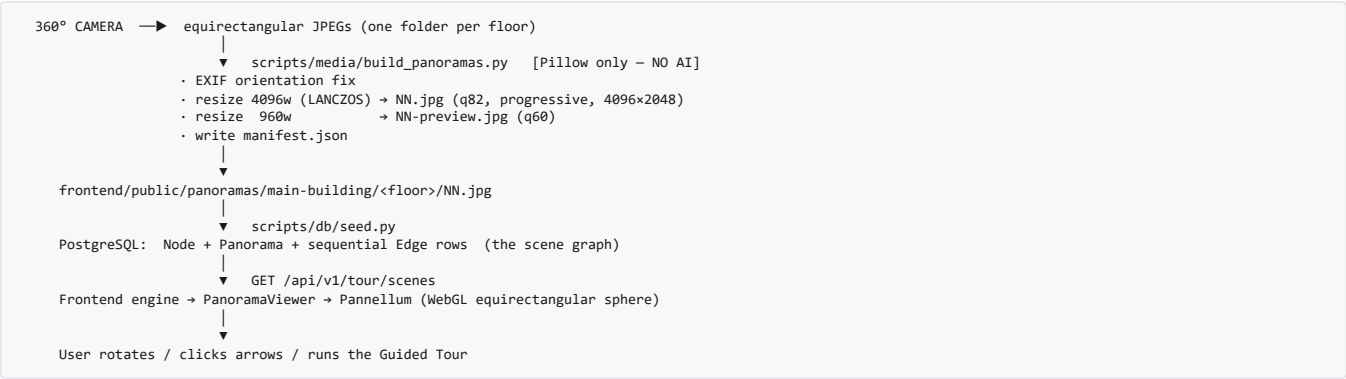
- **Orientation calibration:** an admin-only panel (`NEXT_PUBLIC_ENABLE_ORIENTATION_CALIBRATION`) to set each scene's resting `initial_yaw / initial_pitch` and each edge's `entry_yaw` — done once, then read-only.

17.5 Inventory (verified from `manifest.json` + live DB)

Floor (Main Building)	Level	Real 360° scenes
Entrance	0	23
Ground Floor	1	34
First Floor	2	34
Second Floor	3	31
Third Floor	4	34
Total real Main-Building scenes		156

Plus **5 placeholder panoramas** (`is_placeholder = true` in the DB) for the legacy Phase-5 building set (`Admin`, `BlockA`, `BlockB`, `Library`, `Labs`, ...) that pre-dated the real photos. Live DB: `panoramas` = 161 rows (156 real + 5 placeholder), `nodes` = 180, `edges` = 325.

17.6 Pipeline diagram



18. Virtual Tour Implementation

18.1 Underlying data structure — a graph

Element	DB table	Key fields	Meaning
Node	nodes	id, name, node_type (entrance / junction / corridor / room / staircase / elevator /...), campus_id, building_id?, floor_id?, pos_x?, pos_y?, latitude?, longitude?	a point you can stand at (has a panorama for tour scenes; also a pathfinding vertex)
Edge	edges	source_node_id, target_node_id, distance, is_bidirectional, edge_type (corridor / stairs / elevator / ramp /...), direction (forward / left / right / back / up / down), walking_time, accessible, yaw, hotspot_pitch, label_override, entry_yaw, entry_pitch	a navigable connection; drives both the tour hotspot arrow and the pathfinder
Panorama	panoramas	node_id, image_path, title, is_placeholder, sequence_index, initial_yaw, initial_pitch, hfov (default 110°), description	the 360° image + its calibrated resting view
CrossFloorHotspot	cross_floor_hotspots	source_node_id, target_node_id, yaw, pitch, label	a hand-placed sightline marker, rendered only on its authored scene
Floor	floors	building_id, level, name (unique per building+level)	groups scenes; drives the FloorSelector
Building	buildings	campus_id, name, code, latitude?, longitude?	Main Building + 4 legacy placeholder buildings

- **Weights:** `Edge.distance` (metres — for sequential tour edges this is `SEQUENTIAL_STEP_DISTANCE_M = 4.0`, a documented approximation, since consecutive photos were not surveyed) and `Edge.walking_time`.
- **Directionality:** an edge is stored once; `is_bidirectional = true` makes `build_graph` add the reverse direction too.

18.2 How the frontend uses it

1. `useTourPanoramas` → `GET /api/v1/tour/scenes?building_id=<MAIN>`, `GET /floors`, `GET /cross-floor-hotspots`.
2. `frontend/src/features/tour/engine/` (`buildPanoramaEngine`, `validatePanoramaEngine`, `applyFloorReclassification`, `buildManualTourHotspots`, `toManualTourPanorama`) converts the API rows into `PanoramaNode` objects with resolved `.next` / `.prev` / `.hotspots`.
3. The `/tour` page holds `currentNode` state; `PanoramaViewer` renders it via Pannellum; clicking a hotspot or a sidebar scene calls `onHotspotClick /navigation`, which switches `currentNode` and mirrors it into `tourStore`.
4. `Minimap`, `FloorSelector`, `GuidedTourControls`, `TourModeToggle` all read/write that shared state.

18.3 Tour modes

Mode	Component / hook	Behaviour
Manual Tour	<code>PanoramaViewer</code> + <code>TourControls</code>	click arrows / drag / keyboard; <code>entry_yaw</code> opens each new scene facing the way you walked
Guided Tour	<code>useGuidedTour</code> + <code>GuidedTourControls</code> / <code>GuidedTourPanel</code>	auto-walk the floor's sequence: at each scene it eases the camera left/right/forward, then advances; pause / resume / restart / speed (0.5× – 2×); honours <code>prefers-reduced-motion</code>
Immersive toggle	<code>ImmersiveToggle</code>	hides UI chrome for a full-screen view

18.4 Cross-floor navigation

- Within a floor: the calibrated `forward` / `back` edges form the walk sequence; `Next` / `Previous` continue across the floor boundary.
- Between floors: `staircase` / `elevator` nodes + edges (`direction` = up/down, `edge_type` = stairs/elevator), rendered as ↴ / ↲ / ⤴ hotspots; plus the hand-placed `cross_floor_hotspots` sightline markers.

18.5 Minimap

`Minimap.tsx` — a white-themed top-down schematic of the current floor. A live heading marker is updated every animation frame from `PanoramaViewer`'s `getCurrentView()` (yaw relative to the scene's calibrated `initial_yaw`, so the compass reads 0° at the calibrated view). Purely a projection/visual aid — no routing.

19. Navigation and A* Pathfinding

Status: Implemented (backend engine + HTTP endpoint) · Not currently reachable from the UI · Not used by the chatbot.

19.1 What exists

Piece	Location	Status
Graph builder	backend/app/navigation/graph_builder.py → build_graph(db, accessible_only=False)	Implemented, used
A* search	backend/app/navigation/pathfinding.py → find_shortest_path(graph, start_id, goal_id)	Implemented, tested
Dijkstra (all-pairs from a source)	same file → single_source_distances(graph, start_id)	Implemented
Turn-by-turn formatter	backend/app/navigation/__init__.py re-exports format_directions	Implemented
HTTP endpoint	GET /api/v1/navigate?start_node_id=&destination_node_id=&accessible_only= → backend/app/api/v1/navigate.py	Implemented, registered
Response schema	backend/app/schemas/navigation.py → RouteResponse	Implemented
Frontend API client	frontend/src/api/navigate.ts	Deleted in commit b3219d9 ("remove ... GPS navigation")
Chatbot navigation routing	removed from the chat path in Phase 9	navigation_agent now does location lookups only

So the algorithm is real, correct, and exercised by scripts/ai/test_* navigation scripts and by hitting the endpoint directly, but **no page in the running frontend calls it today** and **no chat message reaches it**.

19.2 Data model behind the graph

- **Vertices** = rows in nodes (live count **180**). Each node optionally carries floor-plan coordinates pos_x, pos_y and/or GPS latitude, longitude.
- **Edges** = rows in edges (live count **325**). Each edge has distance (metres), is_bidirectional, walking_time (seconds), accessible (bool), plus tour-hotspot fields (yaw, hotspot_pitch, ...).
- build_graph turns these into an adjacency list dict[int, list[GraphEdge]]. A bidirectional edge is inserted in both directions; a one-way edge (e.g. a fire-exit stair) only forward. accessible_only=True filters out non-accessible edges before the graph is built, so the search physically cannot route over stairs.

19.3 The A* algorithm as implemented

- **Purpose**: shortest walking route between two campus nodes.
- **Cost so far** g(n) = sum of edge.distance along the path found to n.
- **Heuristic** h(n) = straight-line (Euclidean) distance from n to the goal:
- if both nodes have pos_x/pos_y → hypot(dx, dy) in plan units;
- else if both have lat/long → local planar approximation $dx = \Delta lon \cdot 111320 \cdot \cos(\text{mean_lat})$, $dy = \Delta lat \cdot 110540$, then hypot(dx, dy);
- else → 0.0, which makes A *degrade* to Dijkstra* (still correct, still admissible). There is deliberately no separate Dijkstra code path for this fallback.
- **Priority** f(n) = g(n) + h(n), popped from a binary min-heap (heapq).
- **Admissibility**: straight-line distance never exceeds real walking distance along edges, so the heuristic is admissible ⇒ A* returns an optimal path.

Pseudocode (mirrors find_shortest_path):

```
function find_shortest_path(graph, start, goal):
    if start not in graph.nodes: raise NoPathFoundError
    if goal not in graph.nodes: raise NoPathFoundError
    if start == goal: return PathResult([start], [], 0.0)

    open_heap = [(0.0, start)]          # (f_score, node_id)
    came_from = {}                      # node -> (prev_node, edge)
    g_score = {start: 0.0}
    visited = set()

    while open_heap:
        _, current = heappop(open_heap)
        if current in visited: continue
        visited.add(current)
        if current == goal:
            return reconstruct_path(came_from, start, goal, g_score[goal])
        for ge in graph.adjacency[current]:          # ge = GraphEdge(neighbor, distance, edge)
            tentative_g = g_score[current] + ge.distance
            if tentative_g < g_score.get(ge.neighbor, +inf):
                g_score[ge.neighbor] = tentative_g
                came_from[ge.neighbor] = (current, ge.edge)
                f = tentative_g + heuristic(nodes[ge.neighbor], nodes[goal])
                heappush(open_heap, (f, ge.neighbor))

    raise NoPathFoundError
```

- **Complexity**: $O(E \log V)$ time (each edge relaxation costs one heap push), $O(V + E)$ space. With $V = 180$, $E = 325$ this is effectively instantaneous.
- **Output** (PathResult): node_ids, edges, total_distance; derived properties estimated_walk_time_minutes (sum of edge.walking_time, or distance / $1.2 \text{ m}\cdot\text{s}^{-1}$ if no edges) and is_accessible (every edge wheelchair-passable).
- format_directions converts consecutive edges into human strings using each edge's direction enum (forward/left/right/back/up/down) and edge_type.
- RouteResponse fields: path (node names), total_distance, estimated_walk_time, turn_by_turn.

19.4 `single_source_distances` (Dijkstra)

Plain Dijkstra from one source to every reachable node — used for nearest-neighbour questions (nearest room, nearest panorama) where there is no single destination for A's *heuristic to aim at*. Here Dijkstra is the directly appropriate* algorithm, not a fallback.

19.5 Honest status note

The Phase 9 change and the later `b3219d9` map simplification mean this subsystem is currently **dormant**: fully built and tested, wired to an endpoint, but with no user-facing entry point. Re-exposing it (a "route me there" button on the map, or re-attaching it to `navigation_agent`) is listed in the roadmap (§34).

20. Database

Status: Implemented (PostgreSQL + SQLAlchemy 2.0 + Alembic).

20.1 Engines and tooling

Concern	Choice
RDBMS (local dev)	PostgreSQL 18, database <code>gat_campus_tour</code>
RDBMS (Docker)	<code>postgres:16-alpine</code> service <code>db</code>
ORM	SQLAlchemy 2.0 (<code>Mapped</code> / <code>mapped_column</code> typed style)
Migrations	Alembic 1.13, <code>database/migrations/</code> , 12 revision files
Driver	<code>psycopg2-binary</code>
Runtime plumbing	<code>backend/app/db/</code> (engine, session factory, declarative <code>Base</code>)
Vector store (separate)	ChromaDB <code>PersistentClient("./data/chroma_db")</code> , collection <code>gat_kb</code> — not in PostgreSQL

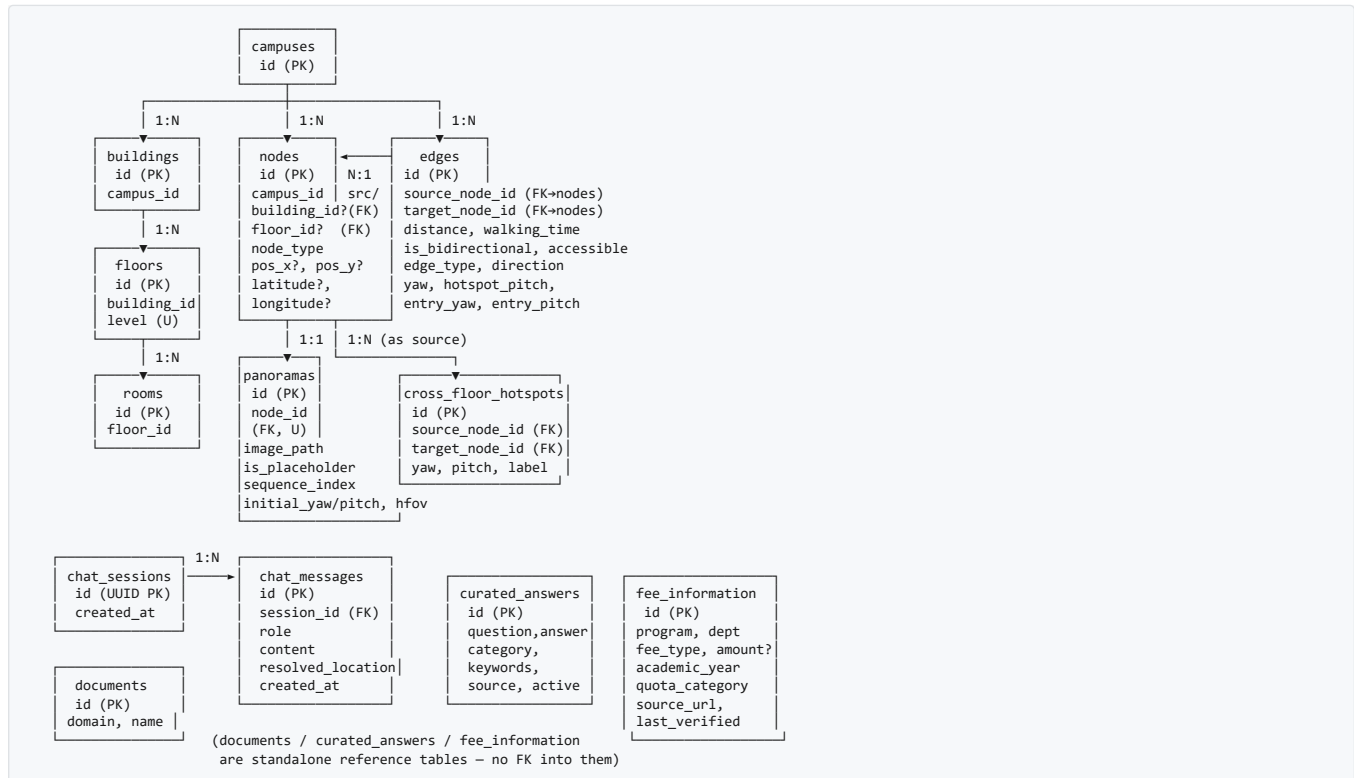
Connection string comes from `DATABASE_URL` (see §27). The chat request handler is **synchronous** and uses a normal scoped `Session` per request via the `get_db` dependency.

20.2 Tables (13 application tables + `alembic_version`)

#	Table	Purpose	Live row count
1	<code>campuses</code>	top-level campus record	1
2	<code>buildings</code>	buildings on a campus (lat/long nullable)	5
3	<code>floors</code>	floors within a building (unique <code>building_id</code> + <code>level</code>)	10
4	<code>rooms</code>	rooms within a floor	11
5	<code>nodes</code>	tour scene points / pathfinding vertices	180
6	<code>edges</code>	navigable connections / tour hotspots	325
7	<code>panoramas</code>	360° image record per node	161
8	<code>cross_floor_hotspots</code>	hand-placed cross-floor sightline markers	849
9	<code>documents</code>	ingested source-document registry	5
10	<code>chat_sessions</code>	one row per assistant session (server UUID, no auth)	474
11	<code>chat_messages</code>	user + assistant turns, <code>resolved_location</code> on assistant rows	1622
12	<code>curated_answers</code>	fallback Q/A tier, matched by MiniLM cosine ≥ 0.55	7
13	<code>fee_information</code>	structured fee rows from GAT Admission 2026 + AICTE EoA	16

Row counts are a snapshot of the live dev database at audit time, not fixed data.

20.3 Entity–relationship diagram (as implemented)



- **nodes** is the hub: self-referential many-to-many through **edges**, 1:1 with **panoramas** (one panorama per node, **node_id** unique), 1:N with **cross_floor_hotspots** on both **source_node_id** and **target_node_id**.
- **chat_sessions** → **chat_messages** is the only user-data relationship; there is **no users table and no authentication** anywhere in the schema.
- **documents**, **curated_answers**, **fee_information** are flat reference tables the assistant reads; they hold no foreign keys.
- **node_type** is a `StrEnum` with 13 values (`entrance`, `junction`, `corridor`, `room`, `staircase`, `elevator`, ...); **edge_type** and **direction** are likewise enums.

20.4 Migrations

`database/migrations/versions/` holds 12 Alembic revisions applied in sequence (campus/building/floor/room core → nodes/edges → panoramas → cross-floor hotspots → chat sessions/messages → `resolved_location` column → curated answers → fee information). `alembic.ini` sits at the repo root with `script_location = database/migrations`. Autogenerate + alembic upgrade head is the documented workflow.

21. Voice Assistant

Status: Implemented (browser-native only) · No server-side voice cost.

21.1 Design decision

All speech is **Web Speech API** in the browser. There is no server-side STT/TTS, no audio upload, no cloud speech key. This is a deliberate cost and privacy choice recorded in the build guide (Phase 4).

21.2 Speech-to-text — frontend/src/hooks/useSpeechRecognition.ts

Aspect	Implementation
API	<code>window.SpeechRecognition ?? window.webkitSpeechRecognition</code>
Mic acquisition	<code>acquireMicStream()</code> — tries <code>getUserMedia({audio:true})</code> , then falls back to iterating <code>enumerateDevices()</code> audio inputs by <code>deviceId</code> when the default device fails
Interaction	press-and-hold (push-to-talk) and tap-to-toggle
Safety cap	hard stop at ~25 s so a stuck recogniser can't run forever
Language	follows UI language: <code>en</code> → <code>en-IN</code> , <code>kn</code> → <code>kn-IN</code> , <code>hi</code> → <code>hi-IN</code> (<code>SPEECH_LANG</code> map)
Degradation	if the API is absent the mic button is hidden / disabled; typing still works
Diagnostics	<code>frontend/src/lib/micDiagnostics.ts</code> + <code>MicDiagnosticPanel.tsx</code> — enumerates devices, checks permission state, surfaces errors to the user (not just the console)

21.3 Text-to-speech — frontend/src/hooks/useSpeechSynthesis.ts + voiceSelection.ts

Aspect	Implementation
API	<code>window.speechSynthesis</code> + <code>SpeechSynthesisUtterance</code>
Voice choice	<code>voiceSelection.ts</code> — deterministic : scores available voices, prefers an <code>en-US</code> male voice, tie-broken by a fixed priority list, so the same voice is used every time (fixes an earlier "voice changes between answers" bug)
Persistence	chosen voice URI cached in <code>localStorage</code>
Prosody	<code>rate 0.97</code> , <code>pitch 0.95</code> , <code>volume 1.0</code>
Trigger	"read answers aloud" toggle in the assistant UI; when on, each new assistant message is spoken once (<code>viaVoice</code> guard prevents double-speak)
Language	utterance <code>lang</code> follows the same <code>en-IN</code> / <code>kn-IN</code> / <code>hi-IN</code> selection
Tests	<code>frontend/scripts/test-voice-selection.mts</code> — 16 pure unit tests over the scoring function

21.4 Voice navigation commands

"Take me to X" style phrases are recognised **as text** by the supervisor's `NAVIGATION_PHRASES` list (23 patterns) after STT converts them, and routed to `navigation_agent`, which currently answers with a location description (Phase 9 removed live route-walking from the chat path). The voice layer itself adds no separate command grammar.

22. API Architecture

Status: Implemented (FastAPI, versioned under `/api/v1/`).

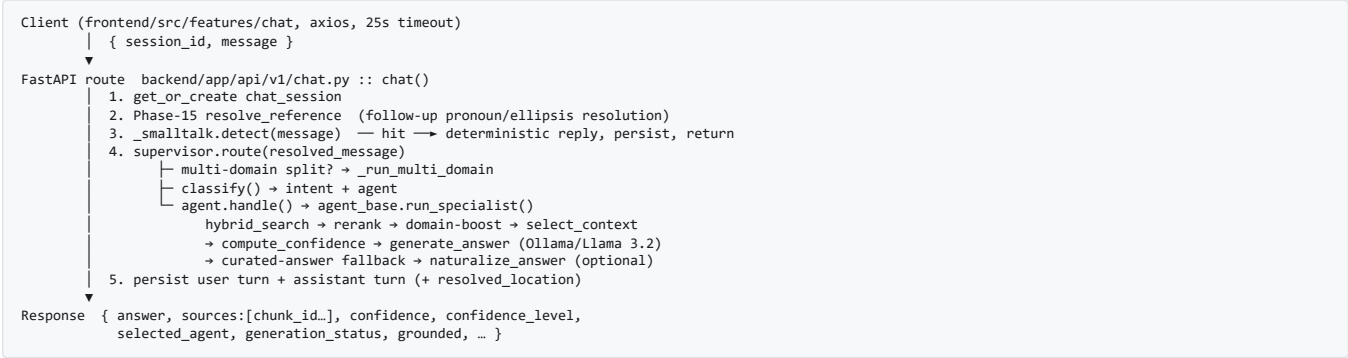
22.1 Registered routers (`backend/app/api/v1/__init__.py`)

Router	Prefix	Methods	Notes
chat	<code>/api/v1/chat</code>	POST	the assistant; synchronous handler; <code>{session_id, message}</code> → <code>{answer, sources, confidence, ...}</code>
campuses	<code>/api/v1/campuses</code>	GET	reference data
buildings	<code>/api/v1/buildings</code>	GET	reference data
floors	<code>/api/v1/floors</code>	GET	reference data
rooms	<code>/api/v1/rooms</code>	GET	reference data
nodes	<code>/api/v1/nodes</code>	GET	graph vertices
edges	<code>/api/v1/edges</code>	GET	graph edges
panoramas	<code>/api/v1/panoramas</code>	GET	360° image records
cross-floor-hotspots	<code>/api/v1/cross-floor-hotspots</code>	GET	tour markers
documents	<code>/api/v1/documents</code>	GET	ingested-source registry
tour	<code>/api/v1/tour/scenes</code>	GET	composed scene payload for the tour UI (<code>?building_id=</code>)
navigate	<code>/api/v1/navigate</code>	GET	A route (see §19); no current frontend caller*
health	<code>/health</code>	GET	liveness probe (defined on the app, not under <code>/api/v1</code>)

22.2 Conventions actually followed

- **Versioning:** everything under `/api/v1/`. A breaking change would get `/api/v1/` → new prefix, not a silent edit.
- **response_model everywhere:** each route declares a Pydantic schema from `backend/app/schemas/`; handlers never return raw dicts.
- **Grounding contract:** every response carrying an AI answer includes `sources` (chunk IDs). Conversational small-talk replies return `sources: []` and `generation_status: "conversational"`.
- **CORS:** restricted to `CORS_ALLOWED_ORIGINS` (the configured frontend origin); never `*`.
- **REST vs streaming:** all endpoints are plain request/response JSON. The build guide reserves WebSocket/SSE for token streaming of the chat answer; **that streaming transport is not implemented yet** — the current `/api/v1/chat` returns the full answer in one response.
- **Error handling:** `backend/app/main.py` registers exception handlers; agent/retriever/LLM failures are caught at the agent boundary and degrade to the refusal string or a typed error, not a raw 500.

22.3 Request lifecycle for `POST /api/v1/chat`



22.4 App startup (`lifespan`)

On boot, `main.py`'s `lifespan` calls `chat.warmup()` — a single throwaway generation against the configured `OLLAMA_MODEL` so the first real user request doesn't pay the model-load latency. CORS, exception handlers, and routers are registered at import time.

23. End-to-End Data Flows

Each flow below is traced from the actual code path.

23.1 Grounded campus question — "Where is the CSE department?"

```
Browser ChatInput → POST /api/v1/chat {session_id, "Where is the CSE department?"} →  
chat() → session ensured  
resolve_reference → "independent" (no prior context needed)  
smalltalk.detect → None  
supervisor.route:  
  detect_multi_domain → single domain  
  classify: NAVIGATION_PHRASES? no · room-number regex? no  
    · DOMAIN_KEYWORDS "department" → academic ; "where" → navigation  
    → navigation wins on phrase position → navigation_agent  
  navigation_agent.handle:  
    spatial_knowledge.search_spatial("CSE department")  
    → data/campus_spatial/spatial_knowledge.json record DEPT_CSE_FIRST_FLOOR  
      (evidence: overhead sign in main-building/first-floor panorama, confidence 0.95)  
    builds answer with "Evidence:" line  
    naturalize_answer (if LLM_NATURALIZE) → llama rephrases, guards re-check numbers/entities  
    persist user + assistant turn (resolved_location = "CSE department / first floor")  
← { answer, sources, confidence=HIGH, selected_agent:"navigation_agent",  
    tool_used:"spatial_knowledge", generation_status:"tool" }  
Browser renders answer; if "read aloud" on → speechSynthesis speaks it once
```

23.2 Retrieval-grounded question — "What is GAT's NAAC grade?"

```
route → classify → DOMAIN_KEYWORDS none strong → LSTM classify_intent  
  → (low/mid conf) → general_agent (or admission_agent if "accreditation" keyword)  
agent_base.run_specialist("What is GAT's NAAC grade?"):  
  query_expansion.expand_query → widened terms  
  HybridRetriever.search:  
    dense: ChromaDB gat_kb cosine top-20 (MiniLM 384-dim query embedding)  
    sparse: BM25Okapi over same corpus top-20  
    normalize each min-max → fuse 0.6·dense + 0.4·bm25 → top candidates  
  Reranker (heuristic_fallback): 0.55·hybrid + 0.25·coverage + 0.10·phrase + 0.10·length  
  apply_domain_boost (+0.05 for agent domain)  
  select_context: drop Jaccard≥0.75 dupes, drop <0.35×top, keep top-5  
  compute_confidence: 0.4·intent + 0.6·retrieval ; retrieval = 0.7·top1 + 0.3·agreement  
  → HIGH (≥0.58)  
  generate_answer: Ollama llama 3.2, SYSTEM_PROMPT ("answer only from context"),  
    context = 5 chunks ; semaphore width 1  
  grounding.find_unsupported_claims(answer, context):  
    "A" grade → not a digit-run ; passes  
  naturalize (optional) → guards pass  
← { answer:"GAT holds an 'A' grade from NAAC.", sources:[chunk_ids...],  
    confidence_level:"HIGH", grounded:true, selected_agent:"general_agent" }
```

23.3 Refused / out-of-scope question — "What is the capital of France?"

```
route → classify → general_agent  
run_specialist:  
  hybrid_search → chunks are all low similarity  
  compute_confidence → retrieval component low → overall < 0.48 → LOW  
  LOW branch: LLM is NOT called ; return fixed refusal  
← { answer:"I can only help with questions about Global Academy of Technology..",  
    confidence_level:"LOW", grounded:false, sources:[] }
```

23.4 Small talk — "Thanks a lot"

```
chat() → smalltalk.detect("Thanks a lot") → "gratitude"  
  build_response("gratitude", RESPONSE_LANGUAGE) → deterministic string  
  NO retrieval, NO BM25, NO DB read beyond session, NO LLM (~30 ms)  
persist turns (generation_status="conversational", selected_agent="conversation_agent")  
← { answer:"You're welcome! ..", sources:[], generation_status:"conversational" }
```

23.5 Multi-domain question — "Tell me about admissions and the CSE department"

```
supervisor.detect_multi_domain → splits on " and " → ["Tell me about admissions",  
  "the CSE department"]  
_run_multi_domain:  
  sub-query 1 → admission_agent.handle → grounded answer A (own retrieval)  
  sub-query 2 → academic_agent.handle → grounded answer B (own retrieval)  
  concatenate with headings ; union of sources ; min() of confidences  
← { answer: "***Admissions**\n...\nCSE Department**\n...", sources:[AUB],  
    selected_agent:"multi_domain", generation_status:"multi_domain" }
```

23.6 Aggregated list — "What departments are available?"

```
academic_agent.handle:  
  _LIST_DEPARTMENTS_PATTERNS matches  
  _aggregate_departments:  
    retrieve chunks whose source_url matches gat.ac.in/_engineering...html  
    collect distinct department pages → CSE, ISE, ECE, EEE, ME, CE  
    naturalize_answer(required_entities=[all six]) – guard fails answer if any dropped  
← { generation_status:"aggregated", grounded:true, sources:[dept-page chunk_ids] }
```

23.7 Contextual follow-up — "Where is it?" after "Tell me about the library"

```
chat() → resolve_reference("Where is it?", prior_turns):  
  finds antecedent "the library" in previous assistant turn's resolved_location  
  → status "resolved", rewritten = "Where is the library?"  
supervisor.route("Where is the library?") → navigation_agent → normal flow
```

23.8 Voice input flow

```
User holds mic button  
  useSpeechRecognition: acquireMicStream → SpeechRecognition(lang = en-IN|kn-IN|hi-IN)  
  interim transcripts shown ; on release → final transcript  
  transcript dropped into ChatInput as if typed → normal POST /api/v1/chat  
Assistant answer returns → if "read aloud" toggle on:  
  useSpeechSynthesis: pick deterministic voice (voiceSelection) → speak once (viaVoice guard)
```

23.9 360° tour scene change

```
/tour page mounts → useTourPanoramas:  
  GET /api/v1/tour/scenes?building_id=<MAIN> + /floors + /cross-floor-hotspots  
  engine/ builds PanoramaNode graph (resolved .next/.prev/.hotspots from edges)  
currentNode state → <PanoramaViewer> (dynamic import, ssr:false)  
  Pannellum loads /panoramas/<image_path> (equirectangular, 4096×2048)  
  hotspots = outgoing edges, placed at (edge.yaw, edge.hotspot_pitch)  
User clicks a hotspot:  
  onHotspotClick(targetNodeId) → setCurrentNode → scene-transition veil  
  new scene opens facing edge.entry_yaw / entry_pitch  
  tourStore mirrors currentNode ; Minimap heading marker follows getCurrentView()
```


24. Implementation Decisions — Why Each Technology

For each: what was chosen, why, and the alternative that was considered/rejected.

Choice	Why (from the code + build guide)	Alternative not taken
Next.js 15 App Router + React 19 + TS strict	one framework for routing, SSR/CSR split, and a large component tree; TS strict catches contract drift between the API client and components	plain React SPA (no routing/SSR story); SvelteKit (smaller ecosystem for the map/pano libs used)
Tailwind CSS	fast iteration on a themed marketing-style campus site without a separate CSS pipeline	CSS Modules / styled-components (more boilerplate per component)
zustand (+ persist)	tiny cross-component stores (<code>tourStore</code> , session, chat) with <code>localStorage</code> persistence and no provider tree	Redux Toolkit (heavier); React Context only (re-render cost, no persistence)
FastAPI + Pydantic 2	typed request/response models double as the API contract and OpenAPI docs; dependency injection for the DB session	Flask (no built-in validation/schema); Django REST (ORM/opinionation not needed)
PostgreSQL + SQLAlchemy 2.0 + Alembic	relational data (campus→building→floor→room, node/edge graph, sessions) is genuinely relational; Alembic gives reproducible schema history	SQLite (concurrency limits for the demo load test); a document DB (the graph + FK integrity fit SQL better)
ChromaDB (embedded, on disk)	zero-ops local vector store, persists to <code>./data/chroma_db</code> , cosine space, good enough for ~1.5k chunks	Pinecone/Weaviate (external service, cost, network); pgvector (would add a Postgres extension dependency)
all-MiniLM-L6-v2 embeddings	384-dim, fast on CPU, strong sentence-similarity baseline, no GPU or API needed	OpenAI <code>text-embedding-3</code> (external paid API, excluded by the stack rules); larger <code>all-mpnet</code> (slower on CPU for marginal gain)
BM25 (rank_bm25) + dense hybrid	dense retrieval misses exact tokens (room numbers, "NAAC", "KCET"); BM25 catches literal matches; 0.6/0.4 fusion balances both	dense-only (misses rare literal terms); BM25-only (misses paraphrases)
Heuristic reranker (SVR present but untrained)	no labelled relevance data exists to train the SVR, so a transparent weighted heuristic (<code>hybrid/coverage/phrase/length</code>) is used and honestly labelled <code>heuristic_fallback</code>	a cross-encoder reranker (heavy on CPU, and still needs eval data to tune)
Ollama + Llama 3.2 (local)	no external API key, no per-token cost, runs on the dev machine, satisfies the "no paid API" stack rule; single <code>OLLAMA_MODEL</code> env var	Claude/OpenAI API (cost, key management, offline demo impossible) — explicitly superseded in CLAUDE.md; Gemini (referenced in some notes, not implemented)
Hand-rolled supervisor router	intent → agent routing is a handful of rules + one small classifier; a graph framework would be dead weight	LangGraph (listed as an optional <i>future</i> upgrade, not a dependency today)
PyTorch LSTM intent classifier	tiny (~75 KB), CPU-instant, demonstrates an RNN in the stack; used only as a routing <i>fallback</i> after keyword rules	a transformer classifier (overkill for 12 coarse classes, more weight); keyword-only routing (no ML component to show)
A* over the campus node/edge graph	optimal shortest path with an admissible straight-line heuristic; degrades cleanly to Dijkstra when coordinates are missing	Dijkstra always (slower with no heuristic); precomputed all-pairs (graph changes with data edits)
Panorama scene-graph (nodes+edges+panoramas in Postgres)	one graph drives the tour hotspots, the pathfinder, and the map markers — single source of truth; swapping in real photos is an asset-only change	per-scene JSON config files (drifts from the pathfinding graph); a game engine scene format (not web-native)
Pannellum (WebGL equirectangular)	mature, small, renders a 2:1 equirectangular sphere directly in the browser, no build step for the viewer	Three.js hand-rolled sphere (more code, was tried for the 3D map and removed); Marzipano (heavier, tiled-image oriented)
Pillow-only panorama build	the source images are already real 360° camera exports; all that's needed is resize + preview + manifest — no CV/AI	a stitching pipeline (Hugin/OpenCV) — unnecessary, the camera already stitched; a flat-to-360 diffusion model — not used, not needed
Web Speech API (browser STT/TTS)	zero server cost, zero audio upload, works offline-ish, degrades gracefully	Whisper server-side (compute cost, latency, privacy); cloud speech APIs (paid, key)
Plain-dict i18n (<code>translations.ts</code>)	three languages, static UI strings only — a full i18n runtime is unnecessary; AI answers are translated by the LLM system prompt instead	i18next/react-i18next (build guide's original pick; the lighter dict was chosen in implementation — noted as a discrepancy in Appendix B)
Google Maps satellite (<code>@vis.gl/react-google-maps</code>)	a real satellite image of the ~10-acre campus with zoom-based labels, no hand-drawn GeoJSON to maintain	MapLibre + custom campus GeoJSON (build guide's original pick; replaced in Phase 17 — Appendix B); Leaflet (similar effort, fewer React bindings)
Docker Compose (frontend/backend/db/ollama)	one command brings up the whole stack including the model runtime for a demo	Kubernetes (massive overkill); bare-metal setup only (no reproducibility)

25. Project Uniqueness — What Makes This Different

- **Fully local AI stack.** No paid API, no cloud key. The LLM (Llama 3.2 via Ollama), the embeddings (MiniLM), the vector DB (ChromaDB), and the intent classifier (PyTorch LSTM) all run on the machine. The whole assistant works offline once the model is pulled.
- **Real 360° photography, not synthesised.** The tour uses actual equirectangular photos of GAT's Main Building (5 floors, 156 real scenes). There is no diffusion/depth/AI panorama generation — and the documentation says so plainly rather than implying a model that doesn't exist.
- **One graph, three consumers.** The `nodes / edges` graph in PostgreSQL is simultaneously: the 360° tour's hotspot layout, the A* pathfinder's search space, and the map's marker placement source. Editing campus data in one place updates all three.
- **Honest grounding, not just a prompt.** Beyond "answer only from context," a post-generation `find_unsupported_claims` check compares digit-runs (phone numbers, years, fees, room numbers) in the answer against the retrieved context and blocks the answer if a number was invented.
- **Deterministic small-talk layer.** Greetings/thanks/farewells never touch retrieval, the DB, or the LLM — they get a fixed multilingual reply in ~30 ms, keeping the "grounded only" contract intact for everything that is a real question.
- **Layered fallback for answers.** curated answers → spatial-knowledge tool → hybrid RAG → refusal, with a confidence score gating whether the LLM is even called.
- **Multi-agent routing that is actually just routing.** The "multi-agent" design is a supervisor + specialists that all share one retriever and one LLM client — not a swarm of independent model instances, and not conflated with concurrency handling.
- **Built phase-by-phase, each demoable.** 19 phases of git history, each an independently runnable increment, with the build discipline recorded in `CLAUDE.md`.

26. Setup and Installation

Status: Implemented — these are the actual commands from `CLAUDE.md`, `requirements.txt`, `package.json`, and `docker-compose.yml`.

26.1 Prerequisites

Tool	Version seen in this project
Python	3.11+ (type-hint syntax <code>X None</code> , <code>StrEnum</code>)
Node.js	18+ (Next.js 15)
PostgreSQL	16 (Docker) / 18 (local dev machine)
Ollama	any recent; must be able to pull <code>llama3.2</code> (~2 GB)
Docker + Docker Compose	only for the containerised path

26.2 Local (non-Docker) setup

```
# 1. Clone and enter
git clone <repo-url> && cd "Virtual Campus"

# 2. Python environment
python -m venv venv
venv\Scripts\activate           # Windows; source venv/bin/activate on macOS/Linux
pip install -r requirements.txt

# 3. Environment files
cp .env.example .env            # then edit SECRET_KEY, POSTGRES_PASSWORD, DATABASE_URL
cp frontend/.env.local.example frontend/.env.local # set NEXT_PUBLIC_API_BASE_URL (+ Google Maps key if used)

# 4. Database
# create the Postgres database named in DATABASE_URL, then:
alembic upgrade head           # run from repo root (alembic.ini lives there)

# 5. Local LLM
ollama pull llama3.2
ollama serve                   # if not already running (default :11434)

# 6. Knowledge base (one-time, builds ./data/chroma_db)
python scripts/ai/collect_website.py # crawl gat.ac.in
python scripts/ai/collect_pdfs.py    # extract GAT PDFs
python scripts/ai/clean_and_chunk.py # → data/processed/chunks.jsonl
python scripts/ai/build_embeddings.py # → ChromaDB collection gat_kb
# (see scripts/ai/ for the exact current entry-point names)

# 7. Optional: seed reference tables
python scripts/db/seed_curated_answers.py
# fee_information + campus/graph/panorama seed scripts live under scripts/db/ and scripts/

# 8. Run
uvicorn app.main:app --reload --app-dir backend # backend on :8000
cd frontend && npm install && npm run dev       # frontend on :3000
```

Note on step 6: `CLAUDE.md` mentions a single `scripts/build_kb.py`; the repository actually implements this as several scripts under `scripts/ai/`. See §13 and Appendix B.

26.3 Docker Compose setup

```
cp .env.example .env # keep POSTGRES_HOST=db and OLLAMA_BASE_URL=http://ollama:11434
docker-compose up --build # brings up: frontend, backend, db (postgres:16-alpine), ollama
# after first boot, pull the model inside the ollama container:
docker-compose exec ollama ollama pull llama3.2
```

`docker-compose.yml` defines four services — `frontend`, `backend`, `db`, `ollama` — with Dockerfiles present for frontend and backend.

26.4 One-command manual startup (Windows / PowerShell) — `scripts/start-project.ps1`

Status: Implemented (added as a developer-experience convenience; changes no application code).

Running only `npm run dev` in `frontend/` starts **just the Next.js frontend** — the backend is a separate process, so `/tour`, `/chat` and every `/api/v1/*` call fail until FastAPI is also running. `scripts/start-project.ps1` starts and health-checks the whole stack:

```
# from the repo root
npm run start:dev           # = powershell -ExecutionPolicy Bypass -File scripts\start-project.ps1
```

What it does, in order:

- Locates the Python venv (`venv\` preferred, else `backend\venv\`).
- Verifies **PostgreSQL** on `127.0.0.1:5432` (starts the `postgres:16` service if stopped).
- Verifies **Ollama** on `127.0.0.1:11434` and that model `llama3.2` is present (starts `ollama serve` if needed; `-SkipOllama` opts out).
- Reports port **3000** (the separate ORCA project) as *not managed* — never inspected as a kill target, never touched.
- Launches **FastAPI** in its own window: `python -m uvicorn app.main:app --app-dir backend --host 127.0.0.1 --port 8000`, then polls `/health` (allowing for the ~20–60 s RAG + LLM warmup).
- Launches **Next.js** in its own window: `npm run dev -- -p 3001`, then waits for the port and the first compile.
- Health checks** (the run fails loudly if any fail): `backend /health → ok`; `frontend / → HTTP 200`; `GET /api/v1/tour/scenes?building_id=<MAIN> → scene count > 0`; a CORS probe with `Origin: http://localhost:3001` echoes that origin; Ollama reachable.

8. Prints the URL summary.

Companion: `scripts/stop-project.ps1` (`npm run stop:dev`) stops only the GAT backend/frontend process trees (matched by the pinned `127.0.0.1:8000 / :3001` sockets and by repo-path command lines), leaving PostgreSQL and Ollama up and never touching port 3000 or Docker.

The backend loads `.env` from the repo root via an absolute path in `backend/app/core/config.py` , so the launch working directory does not matter; `scripts/ai/` is put on `sys.path` from `chat.py` the same way (`Path(__file__)`), also cwd-independent.

26.5 Quality gates

```
# backend
ruff check backend && black backend && mypy backend
pytest                                     # see §28 – currently collects 0 tests

# frontend
cd frontend && npm run lint && npm run type-check
node --loader ... scripts/test-voice-selection.mts # 16 pure voice-selection tests
```

27. Environment Variables

Status: Implemented — this is the full contents of `.env.example` with secret values shown only as placeholders. Never commit a real `SECRET_KEY`, database password, or API key.

27.1 Backend / shared (`.env` at repo root)

Variable	Placeholder / default	Meaning
ENVIRONMENT	development	environment name
BACKEND_HOST	0.0.0.0	uvicorn bind host
BACKEND_PORT	8000	uvicorn port
SECRET_KEY	change-me	app secret — set a real random value in deployment
CORS_ALLOWED_ORIGINS	http://localhost:3000,http://127.0.0.1:3000,...	comma-separated allowed origins; never *
POSTGRES_USER	gat_admin	DB user
POSTGRES_PASSWORD	change-me	DB password — placeholder only
POSTGRES_DB	gat_campus_tour	DB name
POSTGRES_HOST	db (Docker) / localhost (local)	DB host
POSTGRES_PORT	5432	DB port
DATABASE_URL	postgresql+psycopg2://gat_admin:change-me@db:5432/gat_campus_tour	full SQLAlchemy URL
OLLAMA_BASE_URL	http://ollama:11434 (Docker) / http://localhost:11434 (local)	Ollama endpoint
OLLAMA_MODEL	llama3.2	single source of truth for the LLM; read by <code>config.py</code> , <code>llm_generator.py</code> , <code>agent_base.py</code>
LLM_NATURALIZE	true	rephrase verified tool/curated answers via Llama (facts grounding-checked; failure → exact template)
CHROMA_PERSIST_DIR	./data/chroma_db	ChromaDB on-disk path
CHROMA_COLLECTION_NAME	gat_kb	vector collection name

27.2 Frontend (`frontend/.env.local`)

Variable	Placeholder / default	Meaning
NEXT_PUBLIC_API_BASE_URL	http://127.0.0.1:8000/api/v1	backend base URL the browser calls
NEXT_PUBLIC_ENABLE_ORIENTATION_CALIBRATION	false	show the tour orientation-calibration admin panel
NEXT_PUBLIC_ENABLE_CROSS_FLOOR_PLACEMENT	false	show the cross-floor hotspot placement admin panel
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	(blank)	Google Maps JS API key for <code>/map</code> satellite view; blank → graceful "unavailable" state

27.3 Secret-handling notes found in the repo

- `.env.example` ships `SECRET_KEY=change-me` and `POSTGRES_PASSWORD=change-me` — placeholders, not real secrets.
- `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY` is deliberately **blank** in `.env.example`; the comment instructs setting it only in the gitignored `frontend/.env.local` or a deployment secret store, and restricting the key by origin and API.
- No real API keys, tokens, or passwords were found anywhere in the tracked source. (This document contains none.)

28. Testing

Status: Partially implemented — extensive standalone integration scripts; the formal `pytest` suite is empty.

28.1 What the automated runners find

Runner	Configured path	Result
pytest	pyproject.toml → testpaths = ["tests/backend"]	tests/backend/ contains only .gitkeep → 0 tests collected
tests/e2e/ , tests/frontend/	—	only .gitkeep — no e2e or frontend component tests
npm run lint (frontend)	ESLint (Nextjs config)	runs, enforces the frontend lint rules
npm run type-check (frontend)	tsc --noEmit , strict mode	runs, enforces types
frontend/scripts/test-voice-selection.mts	pure TS	16 assertions over the deterministic voice-selection scoring function
ruff / black / mypy (backend)	pyproject.toml	configured; run manually before backend work is "done"

28.2 What actually verifies behaviour — `scripts/ai/test_*.py`

21 standalone scripts (run as `python scripts/ai/test_<name>.py`, not via `pytest`). Each prints `PASS / FAIL` lines and needs the **live stack** (PostgreSQL + ChromaDB + Ollama) because they call the real `supervisor.route()` / `run_specialist()` / endpoint:

```
test_campus_tools.py          test_phase12_semantic_retrieval.py
test_chat_api.py             test_phase13_multi_agent_routing.py
test_chat_persistence.py      test_phase14_grounding_confidence.py
test_hybrid_retrieval.py      test_phase15_contextual_conversation.py
test_llm_generation.py        test_phase16_3d_map.py
test_multi_agent.py           test_phase17_satellite_map.py
test_phase10_nlu_spatial.py   test_phase18_user_location.py
test_phase11_answer_quality.py test_phase19_gps_navigation.py
test_reranking_confidence.py  test_phaseAB_llama_naturalization.py
test_retrieval.py             test_phaseC_academic_dynamic.py
test_smalltalk_conversational.py
```

These cover: hybrid retrieval ranking, reranker + confidence math, multi-agent routing, grounding checks, contextual follow-ups, small-talk isolation, Llama naturalization guards, dynamic department aggregation, GPS→node navigation, and the 3D/satellite map data.

28.3 Honest gaps

- **No CI.** Nothing runs these on push.
- `pytest` **is green but meaningless** — it collects nothing. A viva examiner running `pytest` will see "no tests ran," not a pass.
- **The `scripts/ai/test_*` scripts are not isolated** — they hit a live database and a running model, so results depend on the current KB contents and model version; they are not deterministic unit tests.
- **No frontend component/hook tests** beyond the one voice-selection file.
- **The intent classifier is the only thing with a formal evaluation** (`evaluation/` — see §29).
- `test_smalltalk_conversational.py` **changed on disk during this documentation pass** (test count reduced/renamed) — the current file is authoritative.

29. Performance

Status: No formal performance benchmark was found in the current project for the RAG assistant.

29.1 What does NOT exist (do not cite numbers for these)

- No latency benchmark for `/api/v1/chat` (no timing harness committed).
- No RAG quality metrics — **no RAGAS, no faithfulness/answer-relevancy scores, no precision/recall/F1 for retrieval**. CLAUDE.md's Phase 7 `eval/run_eval.py` + `eval/test_questions.json` + `eval/report.md` **are not implemented** — there is no `eval/` directory.
- No load-test results. `scripts/load_test.py` (CLAUDE.md Phase 3, 50 concurrent sessions) **was not found**.
- No throughput / tokens-per-second measurement for Llama 3.2 on the dev hardware.
- No frontend Lighthouse / Web Vitals report.

29.2 The one real measured evaluation — intent classifier

From `evaluation/intent_classification_report.txt` and `backend/app/intent_model/artifacts/training_metadata.json` (real artifacts in the repo):

Metric	Value
Training examples	134 (80% of 168 synthetic, seeded split)
Validation examples	34
Train accuracy	1.0000 (overfit — expected at this size)
Validation accuracy	0.5294
Macro F1	0.4042
Weighted F1	0.4869
Classes	12
Vocab size	283 tokens
Best epoch	147 / 150

This is why the LSTM is used **only as a routing fallback** after deterministic keyword/phrase rules, and why low classifier confidence (< 0.60) hands off rather than guessing.

29.3 Qualitative / structural performance characteristics (design-level, not benchmarked)

- Small-talk path:** no retrieval, no DB read beyond the session, no LLM → returns in tens of milliseconds by construction.
- LOW-confidence path:** the LLM is **not called** at all — the fixed refusal returns immediately after retrieval scoring.
- LLM concurrency:** a width-1 `threading.Semaphore` serialises Ollama calls, so concurrent chat requests queue at the model; this bounds memory but caps throughput at one generation at a time.
- Startup warmup:** `lifespan` → `chat.warmup()` pays the model-load cost once at boot so the first user request is not slow.
- Vector search scale:** ~1,507 chunks in ChromaDB with an HNSW cosine index — trivially fast at this size.
- Chat handler is synchronous** — under real concurrency the process relies on the uvicorn worker/thread pool, not async I/O (contrary to `docs/architecture.md`'s "all async" aspiration; see Appendix B).

Any latency/accuracy figure not in §29.2 should be treated as **not measured** in this project.

30. Security

Status: Partially implemented — appropriate for a local/demo project, not production-hardened.

Area	Current state
Authentication	None. No users table, no login, no tokens. Sessions are anonymous server-generated UUIDs.
Authorization	None. All read endpoints are public; the two admin panels are gated only by a <code>NEXT_PUBLIC_*</code> build flag (client-side, not a real access control).
CORS	Restricted to <code>CORS_ALLOWED_ORIGINS</code> allow-list; never <code>*</code> .
Secrets	Kept in <code>.env</code> / <code>frontend/.env.local</code> (gitignored). <code>.env.example</code> holds only <code>change-me</code> placeholders and a blank Maps key. No real secret found in tracked source.
SQL injection	SQLAlchemy ORM with bound parameters throughout; no raw string-formatted SQL found.
Input validation	Pydantic schemas validate every incoming request body/query at the API boundary.
Prompt injection	Partially mitigated: the system prompt constrains the LLM to retrieved context, and <code>find_unsupported_claims</code> blocks invented numbers post-generation. Retrieved web content is trusted as-is — a poisoned KB source could influence answers.
LLM data exposure	The LLM runs locally (Ollama); no user message or context leaves the machine. No third-party AI API.
Rate limiting	Not implemented. CLAUDE.md Phase 3 calls for per-session rate limiting; no such middleware was found.
Transport	HTTP in dev; TLS termination is a deployment concern, not configured in-repo.
Logging hygiene	Backend uses <code>logging</code> per module; the Supervisor logs routing decisions (agent, intent, confidence). RAG debug traces log truncated content. No secrets logged.
Dependency risk	All dependencies are open-source and pinned by minimum version in <code>requirements.txt</code> / <code>package.json</code> ; no paid API keys in the dependency surface.

30.1 Notable security-relevant honesty points

- The **admin calibration / hotspot-placement panels** write to the database through real endpoints. They are hidden by a client-side env flag, which is **not** a security boundary — anyone who can reach the API can call those endpoints.
- There is **no rate limiting**, so the `/api/v1/chat` endpoint (which triggers an LLM generation) is trivially abusable in an exposed deployment.
- `SECRET_KEY` is currently unused for session signing (sessions are plain UUIDs); it exists for future auth.

31. Implementation Status Matrix

Legend:  Implemented ·  Partially implemented ·  Documented but not implemented ·  Planned / future ·  Could not be verified

Feature	Status	Primary source	Evidence / notes
Next.js frontend shell (Home/Tour/Assistant/Map/info pages)	✓	frontend/src/app/	routes present and rendering
Themed layout, navbar, footer, language selector	✓	frontend/src/components/layout/	EN/KN/Hi dict-based i18n
Chat assistant UI (input, window, suggested questions, floating widget)	✓	frontend/src/features/chat/	ChatWindow , ChatInput , FloatingAssistant
POST /api/v1/chat grounded RAG answer	✓	backend/app/api/v1/chat.py	synchronous handler
Website + PDF ingestion → chunk → embed → ChromaDB	✓	scripts/ai/	data/processed/chunks.jsonl ≈ 1,507 chunks; collection gat_kb
Dense + BM25 hybrid retrieval (0.6/0.4)	✓	scripts/ai/hybrid_retrieval.py	rank_bm25 , min-max fuse
Query expansion	✓	scripts/ai/query_expansion.py	widens candidate pool only
Reranking	●	scripts/ai/reranker.py	heuristic weighted formula active; SVRReranker untrained — mode always heuristic_fallback
Confidence scoring + thresholds	●	scripts/ai/confidence.py	0.4·intent + 0.6·retrieval ; intent component is not the LSTM — it falls back to the retrieval value (documented gap)
Grounding / unsupported-claim check	✓	scripts/ai/grounding.py	digit-run check for phone/currency/room/year; catches invented numbers , not invented names
LLM answer generation (local)	✓	scripts/ai/llm_generator.py	Ollama + Llama 3.2, width-1 semaphore
Answer naturalization (Phase A/B/C)	✓	scripts/ai/llm_generator.py::naturalize_answer	multi-guard; any failure → exact template; LLM_NATURALIZE flag
Multi-agent supervisor + 6 specialists	✓	scripts/ai/supervisor.py	conversation/admission/academic/facilities/navigation/general
Multi-domain query split	✓	supervisor.detect_multi_domain	splits on " and "/" ;"
Deterministic small-talk layer	✓	scripts/ai/smalltalk.py	8 categories, en/kn/hi, no retrieval/DB/LLM
Contextual follow-up resolution	✓	scripts/ai/conversation_context.py	resolved/ambiguous/no_context/independent
PyTorch LSTM intent classifier	✓ (as fallback)	backend/app/intent_model/	val accuracy 0.5294 , 12 classes; used only after keyword rules
Curated-answer fallback tier	✓	scripts/ai/curated_answers.py	curated_answers table (7 rows), cosine ≥ 0.55
Spatial-knowledge tool (panorama signage facts)	✓	scripts/ai/spatial_knowledge.py	data/campus_spatial/spatial_knowledge.json
Fee information answers	✓	fee_information table (16 rows)	from GAT Admission 2026 + AICTE EoA AY 2026-27
360° virtual tour (Pannellum, real photos)	✓	frontend/src/features/tour/	156 real Main-Building scenes + 5 placeholders
Panorama build pipeline (Pillow)	✓	scripts/media/build_panoramas.py	resize + preview + manifest; no AI/CV
Flat-image → 360° AI conversion	🛑 → not present	—	No dedicated AI model for converting flat images into 360° panoramas was found. Source images are real 360° camera exports.
Guided tour (auto-walk)	✓	frontend/src/features/tour/useGuidedTour	pause/resume/speed, reduced-motion aware
Minimap + floor selector + orientation calibration	✓	frontend/src/features/tour/	calibration panel behind env flag
Cross-floor hotspots	✓	cross_floor_hotspots table (849 rows)	hand-placed sightline markers
A* / Dijkstra pathfinding engine	✓	backend/app/navigation/pathfinding.py	tested; 0(E log V) ; degrades to Dijkstra
GET /api/v1/navigate endpoint	✓	backend/app/api/v1/navigate.py	registered; returns route + turn-by-turn
Frontend caller for navigation	✗ removed	—	frontend/src/api/navigate.ts deleted in commit b3219d9 ; no page calls it
Chatbot-triggered live routing	✗ removed	—	removed from chat in Phase 9; navigation_agent does lookups only
/map Google satellite view	✓	frontend/src/features/mapSatellite/	needs NEXT_PUBLIC_GOOGLE_MAPS_API_KEY ; graceful fallback without
Three.js 3D campus map	✗ removed	—	replaced by satellite view in Phase 17 (b3219d9 removed 3D + deps)
MapLibre GL + custom GeoJSON	🛑	docs/ build guide	build guide's original pick; implemented as Google satellite instead
Voice STT (Web Speech API)	✓	frontend/src/hooks/useSpeechRecognition.ts	push-to-talk + toggle, device fallback, 25s cap
Voice TTS (Web Speech API)	✓	frontend/src/hooks/useSpeechSynthesis.ts + voiceSelection.ts	deterministic voice, localStorage persist
Mic diagnostics panel	✓	frontend/src/lib/micDiagnostics.ts	surfaces device/permission errors to UI
Multi-language UI (EN/KN/Hi)	✓	frontend/src/lib/i18n/translations.ts	plain dict + t(key)
LLM answers in selected language	✓	llm_generator.py _LANGUAGE_INSTRUCTIONS	system-prompt instruction, proper nouns kept
Session persistence	✓	chat_sessions + chat_messages	474 sessions / 1622 messages live; no auth
PostgreSQL + SQLAlchemy + Alembic	✓	backend/app/models/ , database/migrations/	13 tables, 12 revisions

Feature	Status	Primary source	Evidence / notes
Redis session/rate-limit store	 → excluded	docs/architecture.md (stale)	deliberately not in the stack; state is in PostgreSQL
Per-session rate limiting		CLAUDE.md Phase 3	not implemented
Token-by-token answer streaming (WebSocket/SSE)		CLAUDE.md / API conventions	not implemented ; chat returns full answer
Async DB/LLM I/O in the request path		docs/architecture.md	chat handler is synchronous
RAG evaluation pipeline (eval/)		CLAUDE.md Phase 7	not implemented ; no eval/ dir
scripts/load_test.py concurrency test		CLAUDE.md Phase 3	not found
Intent-classifier evaluation		evaluation/	real metrics + confusion matrix
pytest backend suite		tests/backend/	only .gitkeep — 0 tests
scripts/ai/test_*.py integration scripts		scripts/ai/	21 scripts; need live stack; not in CI
Docker Compose full stack		docker-compose.yml	frontend/backend/db/ollama
CI/CD pipeline	 / none found	—	no workflow file located
scripts/build_kb.py + Kaggle CSV	 → not present	CLAUDE.md	no such file, no CSV; KB is a real gat.ac.in crawl

32. Limitations

An honest list, grounded in the audit.

1. **Intent classifier is weak.** 52.9% validation accuracy on 34 examples. It is only safe because it is a *fallback* behind keyword rules and a confidence gate, but a viva examiner will rightly probe this number.
2. **Confidence score's "intent component" is not the classifier.** `confidence.py` documents a `0.4·intent + 0.6·retrieval` blend, but the intent component currently returns the retrieval value — so confidence is effectively retrieval-only.
3. **Reranker SVR is untrained.** The `SVRReranker` class exists but there is no labelled data to fit it; the system always uses the heuristic path. Presented honestly as `heuristic_fallback`.
4. **Grounding check catches numbers, not names.** `find_unsupported_claims` verifies digit-runs (phones, years, fees, rooms). A hallucinated *person's name* or *department fact* with no numbers would pass the check (the system prompt is the only defence there).
5. **No RAG quality evaluation.** No RAGAS, no faithfulness/relevancy scoring, no `eval/` pipeline. Answer quality is asserted by design, not measured.
6. **No performance benchmarks.** No latency, throughput, or concurrency numbers exist.
7. **Chat handler is synchronous** and the Ollama semaphore is width 1 — real concurrent load queues at the model. The `load_test.py` that would expose this was never written.
8. **No authentication or rate limiting.** Fine for a local demo; unsafe to expose publicly (the chat endpoint triggers LLM compute).
9. **Navigation engine is dormant.** A* is implemented and tested but has no UI entry point and is not reachable from chat. "Take me to X" gives a location description, not a walked route.
10. **Tour edge distances are approximate.** Sequential tour edges use `SEQUENTIAL_STEP_DISTANCE_M = 4.0` because consecutive photo positions were not surveyed. Walk-time estimates inherit this approximation.
11. **Coverage is Main Building only.** 156 real scenes cover GAT's Main Building (5 floors). Other buildings are placeholders.
12. **Buildings have no real GPS.** `buildings.latitude/longitude` are nullable and largely NULL; map markers are derived from node plan coordinates, not surveyed positions.
13. **KB freshness.** The knowledge base is a point-in-time crawl of `gat.ac.in` plus a fixed set of PDFs. Nothing re-crawls; stale institutional facts stay stale until the pipeline is re-run.
14. `pytest` **collects nothing.** The real behavioural checks are non-isolated scripts that need a live DB + model, and nothing runs them automatically.
15. **Small-talk detection is regex/keyword based.** Strict full-message coverage avoids false positives, but unusual phrasings of a greeting may fall through to the RAG path (harmless, just slower).
16. **Documentation drift.** Several `CLAUDE.md` / `architecture.md` statements no longer match the code (Kaggle CSV, 5 intent classes, MapLibre, Claude API, Redis, async, chunk sizes). Catalogued in Appendix B.
17. **Google Maps dependency for the map.** Without a key, `/map` degrades to an "unavailable" panel — the campus map is not self-hosted.
18. **LLM is small.** Llama 3.2 (~3B) is chosen for local speed; it is more prone to phrasing errors and misreading context than a larger model, which is part of why so many deterministic guards wrap it.

33. Deployment Readiness

Status: Demo-ready via Docker Compose · Not production-ready.

33.1 What is ready

- `docker-compose up --build` brings up the whole stack (frontend, backend, PostgreSQL, Ollama) with one command.
- Config is fully environment-driven; no hard-coded hosts, ports, or secrets in application code.
- Database schema is reproducible via `alembic upgrade head`.
- CORS is an explicit allow-list.
- The frontend degrades gracefully when optional services (Google Maps, Web Speech API) are unavailable.
- Backend errors are caught at the agent boundary and returned as typed responses, not raw 500s.

33.2 What is missing for production

Gap	Needed
TLS / HTTPS	reverse proxy (nginx/Caddy/Traefik) with certificates
Auth on admin endpoints	the calibration/hotspot-placement write endpoints are unauthenticated
Rate limiting	none — the chat endpoint triggers LLM compute per request
Horizontal scaling	width-1 Ollama semaphore + synchronous chat handler cap throughput; would need a model-serving tier and an async handler
Observability	no metrics/tracing/alerting; logging is local only
CI/CD	no pipeline found; no automated tests gate a deploy
Secret management	<code>.env</code> files only; no vault/secret-store integration
Database backups	not configured in-repo
Model runtime sizing	Ollama + Llama 3.2 needs ~2 GB RAM resident plus generation headroom; the compose file does not pin resources
Static asset serving	panoramas (~156 × multi-MB JPEGs) are served from the app; a CDN would be needed at scale
Health checks / restart policies	<code>/health</code> exists; compose restart policies and container healthchecks are not fully specified

33.3 Deployment shape that would work today

A single VM with Docker Compose, an nginx TLS reverse proxy in front of the frontend and backend, the Google Maps key set as a deployment secret, and the KB build run once on the volume. Suitable for an internal demo or a department showcase; not for public multi-tenant traffic.

34. Future Development Roadmap

Priority tiers: **P1** = highest value / lowest risk, **P2** = valuable but larger, **P3** = nice-to-have / research.

34.1 P1 — close the honesty gaps

Item	Why	Effort
Wire the LSTM probability into <code>confidence.py</code> 's intent component	the blend is documented but not real; makes confidence meaningful	small
Build the <code>eval/</code> pipeline (CLAUDE.md Phase 7) — 25 fixed questions, LLM-as-judge faithfulness/relevancy, <code>results.csv</code> + <code>report.md</code>	gives the project a real quality number instead of "trust the design"	medium
Write <code>scripts/load_test.py</code> (50 concurrent asyncio sessions)	exposes the width-1 semaphore / sync-handler ceiling with data	small
Move <code>scripts/ai/test_*.py</code> behaviour checks into <code>tests/backend/</code> with fixtures + a CI workflow	<code>pytest</code> currently proves nothing	medium
Re-expand the intent training set beyond 168 synthetic examples (log real user queries → label)	52.9% val accuracy is the weakest metric in the project	medium
Add per-session rate limiting on <code>/api/v1/chat</code>	the endpoint triggers LLM compute; trivially abusable if exposed	small

34.2 P2 — feature depth

Item	Why	Effort
Re-attach navigation: a "Route me there" action on <code>/map</code> and/or <code>navigation_agent</code> → <code>/api/v1/navigate</code>	A* is built and tested but unreachable	medium
Token-by-token answer streaming over WebSocket/SSE	the API conventions reserve this; long answers feel slow returned whole	medium
Make the chat handler async (async SQLAlchemy session, async Ollama client) + a real model-serving tier	current sync + width-1 semaphore caps concurrency	large
Extend real 360° coverage beyond Main Building	156 scenes cover one building; the rest are placeholders	large (photography)
Survey real edge distances / building GPS	replace <code>SEQUENTIAL_STEP_DISTANCE_M = 4.0</code> and NULL lat/long with measured values	medium
Scheduled KB re-crawl + incremental re-embed	the KB is a frozen point-in-time snapshot	medium
Add a name/entity grounding check (not just digit-runs)	hallucinated names currently pass <code>find_unsupported_claims</code>	medium
Auth on the admin calibration / hotspot-placement endpoints	currently gated only by a client-side env flag	small

34.3 P3 — research / polish

Item	Why
Train the <code>SVRReranker</code> (or swap in a small cross-encoder) once eval data exists	replace <code>heuristic_fallback</code> with a learned reranker
Self-hosted map tiles (MapLibre + real campus GeoJSON, per the original build guide)	remove the Google Maps key dependency
LangGraph migration of the supervisor	CLAUDE.md lists it as an optional future upgrade
Larger local model option (Llama 3.1 8B / Qwen) behind <code>OLLAMA_MODEL</code>	fewer phrasing errors if hardware allows
Voice command grammar for tour control ("go upstairs", "turn around")	currently only "take me to X" text patterns
Accessibility pass (WCAG) on the tour + chat UI	not audited
Analytics dashboard over <code>chat_messages</code> (top questions, refusal rate, confidence distribution)	the data is already being stored

35. Viva / Interview Preparation

35.1 The 30-second explanation

"GAT AI Virtual Campus is a web app that lets you explore Global Academy of Technology through real 360° photos and ask an AI assistant questions about the college. The assistant uses Retrieval-Augmented Generation: it searches a knowledge base built from the official GAT website and PDFs, then a local Llama 3.2 model writes an answer using only what it found — and refuses if it can't ground the answer. Everything runs locally: no paid API."

35.2 The 1-minute explanation

"The frontend is Next.js. The backend is FastAPI with PostgreSQL. The AI assistant works like this: a user message first hits a small-talk filter for greetings, then a supervisor routes it to one of six specialist agents using keyword rules plus a small PyTorch LSTM intent classifier as a fallback. The chosen agent runs hybrid retrieval — dense vector search in ChromaDB using MiniLM embeddings, combined with BM25 keyword search — reranks the results with a heuristic, computes a confidence score, and if confidence is high enough, sends the top 5 chunks to a local Llama 3.2 model via Ollama. After generation, a grounding check verifies that any numbers in the answer actually appear in the retrieved context. The 360° tour is separate: real equirectangular camera photos rendered by Pannellum, with the scene connections stored as a node/edge graph in PostgreSQL — the same graph an A* pathfinder can walk."

35.3 The 3-minute explanation

Add to the above: - **Data source:** a real crawl of `gat.ac.in` (~60 HTML pages, depth ≤ 2) plus ~135 official GAT PDFs, cleaned with BeautifulSoup, chunked at 800 characters with 120 overlap using LangChain's `RecursiveCharacterTextSplitter`, SHA-256 de-duplicated, embedded with `all-MiniLM-L6-v2` (384-dim, cosine), stored in a ChromaDB collection `gat_kb` (~1,507 chunks). - **Why hybrid retrieval:** dense search understands paraphrases but misses exact tokens like room numbers or "KCET"; BM25 catches literal matches. Scores are min-max normalised and fused $0.6 \text{ dense} / 0.4 \text{ BM25}$. - **Confidence gating:** $0.4 \cdot \text{intent} + 0.6 \cdot \text{retrieval}$, where $\text{retrieval} = 0.7 \cdot \text{top-1 rerank score} + 0.3 \cdot \text{agreement}$. Below 0.48 the LLM is never called — the user gets a fixed refusal. - **Naturalization:** verified tool/curated answers are optionally rephrased by Llama, but bidirectionally grounding-checked so no fact is added or dropped; any guard failure returns the exact template. - **Multi-agent ≠ concurrency:** all six agents share one retriever and one LLM client — it's a routing design, not parallel model instances.

35.4 The 5-minute explanation

Add: - **Small-talk layer:** `smalltalk.py` — 8 regex categories with strict full-message coverage, deterministic replies in English/Kannada/Hindi, no retrieval / DB / LLM, ~30 ms. - **Contextual follow-ups:** "Where is it?" after "Tell me about the library" — `resolve_reference` finds the antecedent from the previous assistant turn's `resolved_location` and rewrites the query. - **Intent classifier honesty:** `Embedding(→32) → LSTM(32) → Dropout(0.2) → Linear(→12)`, trained on 168 hand-authored synthetic examples, **52.9% validation accuracy** — used only as a routing fallback after keyword rules, with a 0.60 confidence floor. - **360° pipeline:** Pillow only — `exif_transpose`, resize to 4096-wide primary + 960-wide preview, write `manifest.json`. **No AI, no diffusion, no depth estimation, no stitching, no flat-to-360 conversion.** The source images are already real 360° camera exports. - **Pathfinding:** A over the `nodes / edges` graph, $f = g + h$, h = straight-line distance (Euclidean on plan coords, planar approximation on GPS, else 0 → degrades to Dijkstra). $O(E \log V)$ with $V=180$, $E=325$. Currently has no UI caller. - **Grounding:** regex-extracts phone/currency/room/year patterns and checks digit-runs ≥ 3 against the context; blocks the answer on a mismatch. Catches invented numbers, not invented names.

35.5 The 10-minute explanation

Cover everything above plus: the four-tier backend architecture (API → agents → domain services → data), the 13-table schema with `nodes` as the hub, the fallback tiers (curated → spatial tool → hybrid RAG → refusal), the deployment story (Docker Compose, 4 services), the honest limitations (§32), and the documented-vs-implemented discrepancies (Appendix B). Walk through one full request trace end to end (§23.2).

35.6 Question bank (30+)

Architecture

- What does "RAG" mean here and why use it? — Retrieval-Augmented Generation: retrieve relevant text from a knowledge base, then have the LLM answer using only that text. It keeps answers grounded in GAT's real content and lets the model refuse when nothing matches, instead of hallucinating.
- Why a local LLM instead of an API? — No per-token cost, no API key, works offline, satisfies the project's "no paid API" rule. Trade-off: Llama 3.2 (~3B) is smaller and needs more guardrails.
- What exactly is "multi-agent" here? — A supervisor that routes a query to one of six specialist handlers (conversation, admission, academic, facilities, navigation, general). They all share one retriever and one LLM client. It is a routing pattern, not parallel agents and not the concurrency mechanism.
- How is concurrency handled then? — Weakly: the chat handler is synchronous and a width-1 semaphore serialises Ollama calls. This is a known limitation; the planned load test was never written.
- Walk me through a request. — See §23.2: route → classify → hybrid retrieve → rerank → domain boost → select context → confidence → (if high enough) Llama generate → grounding check → optional naturalize → persist → respond with sources.
- Four-tier backend? — API routers → agents/orchestration → domain services (`rag/`, `navigation/`, `intent_model/`, `llm/`) → data/infra. A layer only calls the one below it.

Retrieval & RAG

- Which embedding model, and its dimension? — `all-MiniLM-L6-v2`, 384-dim, cosine similarity, normalized embeddings. Runs on CPU, no API.
- Why hybrid retrieval? — Dense misses exact literal tokens (room numbers, "NAAC", "KCET"); BM25 misses paraphrases. Fuse both: $0.6 \cdot \text{dense_norm} + 0.4 \cdot \text{bm25_norm}$.
- How are the two score scales combined fairly? — Each is min-max normalised to [0,1] over the candidate set before the weighted sum.
- Chunking parameters? — `RecursiveCharacterTextSplitter`, `chunk_size=800` chars, `chunk_overlap=120`, separators `["\n\n", "\n", ".", " ", " "]`, then SHA-256 dedup.
- How many chunks, in what store? — ~1,507 chunks in ChromaDB collection `gat_kb`, HNSW cosine index, persisted to `./data/chroma_db`.
- What is the reranker doing? — A weighted heuristic: $0.55 \cdot \text{hybrid} + 0.25 \cdot \text{query-term-coverage} + 0.10 \cdot \text{exact-phrase} + 0.10 \cdot \text{length-score}$. An `SVRReranker` exists but is untrained, so the mode is always `heuristic_fallback`.

13. *How does the system decide to refuse?* — Confidence `< 0.48` (LOW) → fixed refusal string, LLM not called. Also: grounding-check failure after generation → refusal.
14. *What is the confidence formula?* — `0.4·intent + 0.6·retrieval`; `retrieval = 0.7·top1_rerank + 0.3·agreement`. Thresholds HIGH ≥ 0.58, MEDIUM ≥ 0.48. Fitted to a small 9-query sample — a known weakness.
15. *Is the intent classifier in the confidence score?* — Documented as yes, but in code the intent component falls back to the retrieval value. Honest gap; on the roadmap to fix.
16. *What stops hallucinated facts?* — (a) system prompt: answer only from context; (b) `find_unsupported_claims`: any ≥3-digit number in the answer must appear in the context. Limitation: invented names with no digits pass.
17. *Where does the knowledge come from — be specific.* — A real crawl of `https://www.gat.ac.in/` (depth ≤ 2, ≤ 80 pages, ~60 HTML) plus ~135 official GAT PDFs. **No Kaggle CSV** despite what `CLAUDE.md` says.
18. *What's the system prompt?* — It instructs the model to answer strictly from the provided context, keep proper nouns (GAT, VTU, department names) untranslated, answer in the user's selected language, and say it doesn't know rather than guess.

Intent classifier / NLP

19. *Architecture of the LSTM?* — `Embedding(vocab, 32) → LSTM(32→32, 1 layer) → Dropout(0.2) → Linear(32→12)`, packed sequences, softmax over 12 intents.
20. *Training data and accuracy?* — 168 hand-authored synthetic examples, seeded 80/20 split (134/34). **Validation accuracy 0.5294**, macro-F1 0.4042.
21. *Why is that acceptable?* — It's only a fallback after deterministic keyword/phrase rules, and only trusted above 0.60 confidence; otherwise routing falls to `general_agent`.
22. *CLAUDE.md says 5 intent classes — true?* — No; the implemented model has 12. Documentation drift (Appendix B).
23. *How is small talk detected?* — `smalltalk.py`: 8 regex categories, strict full-message coverage so "Hi, where's the library?" is NOT small talk. Deterministic reply, no pipeline.

Virtual tour / 360°

24. *Do you use an AI model to make the 360° images?* — **No. No dedicated AI model for converting flat images into 360° panoramas exists in this project.**
The photos are real equirectangular 360° camera exports; Pillow only resizes and compresses them.
25. *What renders the panorama?* — Pannellum (WebGL), a 2:1 equirectangular texture on a sphere, dynamically imported client-side (`ssr:false`).
26. *How do you move between scenes?* — Each node's outgoing graph edges become clickable hotspots placed at the edge's `yaw / hotspot_pitch`; clicking one switches the scene and opens it facing `entry_yaw`.
27. *How many real scenes?* — 156 real Main-Building scenes (entrance 23, ground 34, first 34, second 31, third 34) + 5 placeholders.
28. *Guided tour?* — `useGuidedTour` auto-walks a floor's sequence, easing the camera at each scene, with pause/resume/speed and `prefers-reduced-motion` support.

Navigation / algorithms

29. *Where is A and is it used?* — `backend/app/navigation/pathfinding.py::find_shortest_path`. Implemented and tested, exposed at `GET /api/v1/navigate`, but no frontend calls it* (client deleted in `b3219d9`) and the chatbot doesn't trigger it (removed Phase 9).
30. *A heuristic and admissibility?** — Straight-line distance to the goal; never overestimates real walking distance ⇒ admissible ⇒ optimal path. Falls back to `h=0` (Dijkstra) when nodes lack a shared coordinate frame.
31. *Complexity?* — $O(E \log V)$ time, $O(V+E)$ space; $V=180$, $E=325$.
32. *Why store the tour as a graph?* — One structure serves three consumers: tour hotspots, the pathfinder, and map markers.

Database / infra

33. *Why PostgreSQL and not a vector DB for everything?* — The relational data (campus→building→floor→room, the node/edge graph, sessions) is genuinely relational with FK integrity. Vectors live separately in ChromaDB.
34. *Redis?* — Not used. Earlier drafts had it; the approved stack keeps session state in PostgreSQL.
35. *How is session state kept?* — `chat_sessions` (server UUID, no auth) + `chat_messages` (both turns, `resolved_location` on assistant rows).
36. *Migrations?* — Alembic, 12 revisions, `alembic upgrade head` from repo root.

Testing / honesty

37. *Does pytest pass?* — It runs but collects 0 tests (`tests/backend/` is empty). Real behavioural checks are 21 `scripts/ai/test_*.py` scripts needing a live DB + model, not in CI.
38. *Any performance numbers?* — None for the RAG assistant. The only formal evaluation is the intent classifier (52.9% val accuracy). Do not cite latency/RAGAS figures — they don't exist here.
39. *Biggest weaknesses?* — Intent accuracy, no RAG eval, no load test, dormant navigation, synchronous handler, no auth/rate-limiting. (§32)

36. Beginner-Friendly Explanation

Imagine you are a new student visiting Global Academy of Technology's website.

The virtual tour. Someone walked through the Main Building with a 360° camera — the kind that photographs everything around it at once — and took photos at 156 spots across five floors. The website shows you one photo at a time; you can drag to look around in a full circle, like Google Street View. Little arrows float in the image; click one and you "walk" to the next spot. The computer knows which spot connects to which because we drew a map of the building as dots (places you can stand) joined by lines (steps between them). That same dot-and-line map could also compute the shortest walk from one room to another using a classic algorithm called A* — that part is built but not switched on in the current website.

The AI assistant. There is a chat box. You type a question like "How do I apply for CSE?" Here is what happens, in plain terms:

1. **Is it just chit-chat?** If you said "hi" or "thanks," the assistant replies instantly with a friendly line and does nothing else.
2. **Which topic?** The assistant guesses whether your question is about admissions, academics, facilities, directions, or something general — mostly by looking for keywords, with a tiny neural network as a backup guesser.
3. **Look it up.** The assistant has already read the entire official GAT website and its PDF brochures and chopped them into ~1,500 small paragraphs. It finds the 5 paragraphs most relevant to your question — using both "meaning" matching and "exact word" matching together.
4. **Is it confident?** It scores how well those paragraphs actually answer you. If the score is too low (say you asked about the weather in Paris), it politely says it can only help with GAT questions — and never makes something up.
5. **Write the answer.** If confident, it hands those 5 paragraphs to a language model (Llama 3.2) running on the same computer — no internet AI service — and asks it to answer *using only those paragraphs*.
6. **Double-check.** Before showing you the answer, it checks that any phone number, fee, year, or room number in the answer actually appeared in the source paragraphs. If not, it refuses rather than risk a wrong number.
7. **Optionally read it aloud** in English, Kannada, or Hindi, using your browser's built-in text-to-speech.

The key idea: the assistant is not "a chatbot that knows things." It is a librarian. It only tells you what it can find in GAT's own documents, it shows you which documents it used, and it says "I don't know" when it can't find an answer.

37. Source-Code Traceability Matrix

Where each major capability lives.

Capability	File(s)	Key symbol(s)
App entry / startup / warmup	backend/app/main.py	lifespan, create_app, register_exception_handlers
Chat endpoint	backend/app/api/v1/chat.py	chat(), warmup(), _resolve_followup_message, _log_rag_debug_trace
Router registry	backend/app/api/v1/__init__.py	api_router includes chat/campuses/.../navigate
Navigation endpoint	backend/app/api/v1/navigate.py	get_route()
Supervisor / routing	scripts/ai/supervisor.py	route(), classify(), detect_multi_domain(), _run_multi_domain(), NAVIGATION_PHRASES, DOMAIN_KEYWORDS
Small talk	scripts/ai/smalltalk.py	detect(), build_response(), _CATEGORY_PATTERNS, _RESPONSES, CONVERSATION_AGENT
Specialist agents	scripts/ai/{conversation,admission,academic,facilities,navigation,general}_agent.py	handle()
Shared agent pipeline	scripts/ai/agent_base.py	run_specialist(), DEFAULT_AGENT_MODEL, _REFUSAL_REASONS
Department aggregation	scripts/ai/academic_agent.py	_aggregate_departments(), _LIST_DEPARTMENTS_PATTERNS, _DEPARTMENT_PAGE_PATTERN
Contextual follow-ups	scripts/ai/conversation_context.py	resolve_reference()
Hybrid retrieval	scripts/ai/hybrid_retrieval.py	HybridRetriever, search(), DENSE_WEIGHT, BM25_WEIGHT, _normalize(), _tokenize()
Query expansion	scripts/ai/query_expansion.py	expand_query()
Reranking	scripts/ai/reranker.py	Reranker, SVRReranker, HEURISTIC_WEIGHTS, .mode
Context selection	scripts/ai/context_selection.py	select_context(), apply_domain_boost(), DUPLICATE_JACCARD_THRESHOLD, RELATIVE_SCORE_FLOOR
Confidence	scripts/ai/confidence.py	compute_confidence(), _intent_component(), _retrieval_component(), thresholds
Grounding check	scripts/ai/grounding.py	find_unsupported_claims(), _PHONE_PATTERN, _CURRENCY_PATTERN, MIN_DIGIT_RUN_LENGTH
LLM generation + naturalization	scripts/ai/llm_generator.py	generate_answer(), naturalize_answer(), SYSTEM_PROMPT, _ollama_semaphore, RESPONSE_LANGUAGE, _LANGUAGE_INSTRUCTIONS
Curated answers	scripts/ai/curated_answers.py	find_curated_answer(), SIMILARITY_THRESHOLD
Spatial knowledge tool	scripts/ai/spatial_knowledge.py	search_spatial(), extract_room_number(), LOW_CONFIDENCE_THRESHOLD
KB ingestion	scripts/ai/collect_website.py, collect_pdfs.py	website crawler, pypdf per-page extraction
KB clean / chunk	scripts/ai/clean_and_chunk.py	CHUNK_SIZE=800, CHUNK_OVERLAP=120, RecursiveCharacterTextSplitter, SHA-256 dedup
KB embed	scripts/ai/build_embeddings.py	SentenceTransformer("all-MiniLM-L6-v2"), PersistentClient, gat_kb
Intent model	backend/app/intent_model/model.py, classify.py, dataset.py, train.py	IntentLSTM, classify_intent(), TRAINING_EXAMPLES, INTENTS
Intent artifacts	backend/app/intent_model/artifacts/	intent_lstm.pt, vocab.json, intents.json, training_metadata.json
Intent evaluation	evaluation/intent_classification_report.txt	accuracy 0.5294, confusion matrix
Pathfinding	backend/app/navigation/pathfinding.py	find_shortest_path() (A*), _heuristic(), single_source_distances() (Dijkstra), PathResult
Graph builder	backend/app/navigation/graph_builder.py	build_graph(), CampusGraph, GraphEdge
ORM models	backend/app/models/*.py	Campus, Building, Floor, Room, Node, Edge, Panorama, CrossFloorHotspot, Document, ChatSession, ChatMessageRecord, CuratedAnswer, FeeInformation
Config	backend/app/core/config.py	Settings, OLLAMA_MODEL, DATABASE_URL, CORS_ALLOWED_ORIGINS
Panorama build	scripts/media/build_panoramas.py	_process_photo(), PRIMARY_WIDTH=4096, PREVIEW_WIDTH=960, manifest.json
Panorama viewer	frontend/src/features/tour/PanoramaViewer.tsx	Pannellum dynamic import, HOTSPOT_META, PanoramaViewerHandle
Tour engine	frontend/src/features/tour/engine/	buildPanoramaEngine, buildManualTourHotspots, applyFloorReclassification

Capability	File(s)	Key symbol(s)
Guided tour	frontend/src/features/tour/useGuidedTour.ts	auto-walk state machine
Minimap	frontend/src/features/tour/Minimap.tsx	live heading from <code>getCurrentView()</code>
Satellite map	frontend/src/features/mapSatellite/	<code>@vis.gl/react-google-maps</code> , <code>SatelliteMapUnavailable</code> , <code>campusLayout.deriveBuildingPlacements</code>
Speech-to-text	frontend/src/hooks/useSpeechRecognition.ts	<code>acquireMicStream()</code> , <code>SpeechRecognition</code>
Text-to-speech	frontend/src/hooks/useSpeechSynthesis.ts, frontend/src/hooks/voiceSelection.ts	deterministic voice scoring, <code>localStorage</code> <code>persist</code>
Mic diagnostics	frontend/src/lib/micDiagnostics.ts, frontend/src/features/chat/MicDiagnosticPanel.tsx	device enumeration, permission state
Chat UI	frontend/src/features/chat/	<code>ChatWindow</code> , <code>ChatInput</code> , <code>FloatingAssistant</code>
i18n	frontend/src/lib/i18n/translations.ts	<code>t(key)</code> , EN/KN/HI dict
State stores	frontend/src/store/	<code>tourStore</code> , session/chat stores (zustand + persist)

38. Conclusion

GAT AI Virtual Campus is a working, phase-built demonstration of a **fully local AI assistant** and a **real 360° campus tour** for Global Academy of Technology.

What is genuinely delivered: - A Next.js frontend with a working assistant, a real 156-scene 360° tour of the Main Building, a satellite map, voice input/output, and a three-language UI. - A FastAPI backend with a retrieval-augmented assistant that grounds every answer in a knowledge base built from GAT's own website and PDFs, cites its sources, gates the LLM behind a confidence score, checks generated numbers against the source, and refuses cleanly when it cannot help. - A multi-agent routing layer, a deterministic small-talk layer, contextual follow-up resolution, and a small PyTorch LSTM intent classifier. - An A* / Dijkstra pathfinding engine over a PostgreSQL node/edge graph, exposed as an HTTP endpoint. - The whole stack — frontend, backend, PostgreSQL, and the Ollama/Llama 3.2 runtime — reproducible with one `docker-compose up`.

What this document has been careful to state plainly: - **No AI model converts flat images into 360° panoramas.** The source images are real 360° camera exports; only Pillow (resize/compress) and Pannellum (WebGL rendering) are involved. - **The LLM is Ollama running Meta Llama 3.2, locally** — no cloud API, no key, no Gemini. - **The knowledge comes from a real crawl of gat.ac.in plus official GAT PDFs** — there is no Kaggle CSV. - **The embedding model is `all-MiniLM-L6-v2`** (384-dim); the intent model is a 12-class LSTM at **52.9% validation accuracy**; the reranker runs a **heuristic** because its SVR is untrained. - **There is no RAG performance benchmark, no load test, and `pytest` collects no tests.** The navigation engine currently has no UI entry point.

The result is a project that is strong on **honest, layered grounding** and **local-first AI**, with a clear, prioritised roadmap (§34) for closing the gaps between what the older design documents describe and what the code does today.

Appendix A — Diagram Index

#	Diagram	Section
1	High-level system architecture (4-tier)	\$6, \$7.1
2	Frontend ↔ backend interaction	\$7.2, \$22.3
3	Chatbot request flow	\$7.2, \$11
4	RAG pipeline — build-time + query-time	\$12
5	LLM pipeline / how the model fits RAG	\$14
6	Multi-agent routing	\$15
7	Database ER diagram	\$20.3
8	Virtual campus / tour architecture	\$7.3, \$18
9	360° / panorama build + render pipeline	\$7.4, \$17
10	Scene graph (nodes / edges / panoramas)	\$18.1, \$20.3
11	A* navigation flow + pseudocode	\$16 (#24), \$19.3
12	Voice I/O pipeline	\$7.5, \$21, \$23.8
13	End-to-end request traces (9 flows)	\$23
14	Deployment architecture	\$33
15	Source → processing → storage → retrieval → LLM	\$13
16	Confidence / grounding decision flow	\$12, \$23.3

Appendix B — Documented-vs-Implemented Discrepancies

Places where `CLAUDE.md`, `docs/architecture.md`, or the build guide describe something the current code does **not** do. In every case, **the code is authoritative** (per `CLAUDE.md`'s own preamble).

#	Document says	Code actually does	Where
1	<code>scripts/build_kb.py</code> relabels a Kaggle CSV into GAT content	No such file, no CSV. KB is a real crawl of <code>gat.ac.in</code> (~60 HTML) + ~135 GAT PDFs, via several <code>scripts/ai/</code> scripts	\$13, \$26
2	Intent classifier has 5 classes (<code>admissions/academics/facilities/navigation/general</code>)	Implemented LSTM has 12 classes	\$15, \$29
3	360° map uses MapLibre GL JS + custom campus GeoJSON	<code>/map</code> uses Google Maps satellite (<code>@vis.gl/react-google-maps</code>); MapLibre is not a dependency	\$24, \$31
4	(<code>architecture.md</code>) LLM is the Claude API	Ollama + Llama 3.2 , local (<code>CLAUDE.md</code> supersedes <code>architecture.md</code> here)	\$14
5	(<code>architecture.md</code>) Redis for session / rate-limit state	No Redis. Session state in PostgreSQL ; rate limiting not implemented	\$20, \$30
6	(<code>architecture.md</code>) chunks $\approx$ 300 tokens / 50 overlap	800 characters / 120 overlap (<code>RecursiveCharacterTextSplitter</code>)	\$12, \$13
7	(<code>architecture.md</code>) all backend I/O is async	The chat request handler is synchronous ; Ollama calls serialised by a width-1 semaphore	\$22, \$29
8	(<code>architecture.md</code> / build guide) the navigation agent calls pathfinding	Removed from the chat path in Phase 9; <code>navigation_agent</code> does location lookups only. A* is reachable only via <code>GET /api/v1/navigate</code>	\$19, \$31
9	<code>confidence.py</code> docstring: intent component = LSTM P(intent)	Intent component falls back to the retrieval value; the LSTM probability is not wired in	\$12, \$31, \$32
10	<code>reranker.py</code> : SVR-based reranking	<code>SVRReranker</code> is untrained ; <code>Reranker.mode</code> is always <code>heuristic_fallback</code>	\$16, \$31
11	CLAUDE.md Phase 7: <code>eval/run_eval.py</code> , <code>eval/test_questions.json</code> , <code>eval/report.md</code> (RAGAS-style judge)	Not implemented ; no <code>eval/</code> directory	\$29, \$32
12	CLAUDE.md Phase 3: <code>scripts/load_test.py</code> (50 concurrent sessions)	Not found	\$29, \$33
13	CLAUDE.md: <code>pytest</code> per <code>pyproject.toml</code> <code>testpaths</code>	<code>tests/backend/</code> has only <code>.gitkeep</code> → 0 tests collected	\$28
14	CLAUDE.md / API conventions: WebSocket/SSE token streaming for chat	Not implemented; <code>/api/v1/chat</code> returns the full answer in one response	\$22
15	<code>docs/architecture.md</code> predates Phase 16→17: a Three.js 3D map	3D map + <code>three / @react-three/*</code> deps removed in commit <code>b3219d9</code> ; satellite-only	\$24, \$31
16	README: "project skeleton ... no phase implementation has started"	Severely outdated — 19 phases are implemented	\$31
17	<code>POST /api/v1/navigate</code> with <code>{from, to}</code> body (<code>CLAUDE.md</code> contract)	Implemented as <code>GET /api/v1/navigate?start_node_id=&destination_node_id=&accessible_only=</code> (code wins)	\$19, \$22
18	Build guide: <code>i18next / react-i18next</code> for UI strings	Implemented as a plain TypeScript dictionary (<code>translations.ts</code>) with a <code>t(key)</code> helper	\$24

End of document.